Système minimal de publication/souscription en BCM4Java

Cahier des charges du projet CPS 2026

Jacques Malenfant
professeur des universités

© 2026. Ce travail est partagé sous licence CC BY-NC-ND.

version 3.0 du 6/02/2026

Résumé

Ce document définit le cahier des charges du projet de l'unité d'enseignement Composants (CPS) pour son instance 2026. Seul support d'évaluation de l'UE, le projet vise à comprendre les principes d'une architecture à base de composants, l'introduction du parallélisme, la gestion de la concurrence et les problématiques de répartition entre plusieurs ordinateurs connectés en réseau. Des notions d'architecture logicielle, de réutilisation et de configuration pour la gestion de la performance et le passage à l'échelle seront aussi abordées.

L'objectif du projet 2026 est plus précisément d'implanter un prototype simplifié d'un système de publication/souscription, tels que ceux utilisés dans les architectures à micro-services (par exemple, Kubernetes) et plus généralement dans les systèmes répartis. Le projet se développe en quatre étapes permettant d'aborder successivement l'implantation d'un système minimal fonctionnel, puis d'augmenter sa complexité et sa performance par l'utilisation du parallélisme, d'en assurer la répartition sur plusieurs processus (voire sur plusieurs ordinateurs en réseau) et enfin d'en mesurer la performance pour affiner ses paramètres de configuration au déploiement. Une petite application utilisant le système permettra de tester cette implantation.

Rappel des dates et informations importantes (détails à la fin du document)

	Le projet ***doit*** être réalisé en ***Java SE 8***.
28/01/2026	Date limite pour la formation des équipes (2 personnes).
16/02/2026	Audit 1 (5% de la note finale).
8/03/2026	Rendu de code préalable à la soutenance de mi-semestre (format `tgz` ou `zip` *uniquement*).
9-10/03/2026	Soutenance de mi-semestre (semaine des ER1, 35% de la note finale).
30/03/2026	**ATTENTION, modifié !** Audit 2 (5% de la note finale).
10/05/2026	Rendu de code préalable à la soutenance finale (format `tgz` ou `zip` *uniquement*).
11-12/05/2026	Soutenance finale (semaine des ER2, 55% de la note finale).
15-19/06/2026	Soutenance de seconde session (créneau à définir, note remplaçant entièrement celle de 1re session).

1 Généralités

1.1 Systèmes de publication/souscription

Un système de communication par publication/souscription est un intergiciel permettant de créer, envoyer, recevoir et consommer des messages (voir figure 1). L'organe principal du système est le ***courtier*** (en anglais ***broker***) qui gère la transmission des messages entre ses clients. Les clients peuvent endosser deux rôles (non exclusifs l'un de l'autre) : le rôle de ***publieur*** (***publisher***) qui crée et publie des messages et le rôle de ***souscripteur*** (***subscriber***) qui s'abonne, reçoit des messages et les consomme (les lit, les traite, etc.).

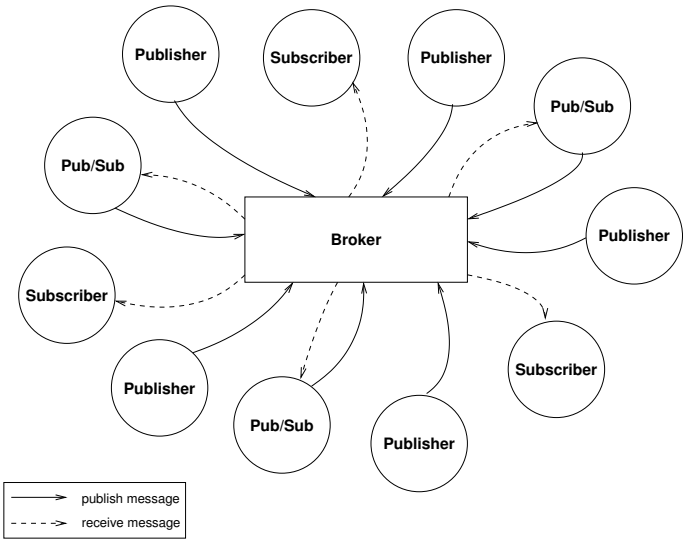


FIGURE 1 – Fonctionnement général d'un système de publication/souscription.

Le modèle de communication par publication/souscription est un modèle de multiplicité 1:N, c'est-à-dire qu'un message publié par un producteur peut être reçu par N consommateurs ($N \geq 0$). Dans ce modèle, les producteurs ne connaissent pas les consommateurs de leurs messages ni les consommateurs le producteur de ceux qu'ils consomment [1] ; le courtier est chargé de faire le lien entre producteurs et consommateurs en recevant les messages publiés et en les acheminant aux souscripteurs. La communication est *asynchrone* : le producteur qui publie un message continue immédiatement son exécution et ne reçoit pas de réponse (de résultat). Le consommateur reçoit les messages correspondant à ses abonnements par un mécanisme similaire à la notification. Le courtier garantit la livraison des messages et le fait que chaque consommateur reçoit chacun des messages qui lui revient une et une seule fois.

La publication et la souscription sont organisées autour de ***canaux*** (*channels*) (aussi appelés sujets *i.e.*, *topics*) : les producteurs publient des messages sur des canaux et les consommateurs souscrivent à la réception de messages sur des canaux. Les consommateurs fournissent à la souscription un ***filtre*** de messages permettant de préciser les messages qui les intéressent parmi les messages publiés sur les canaux auxquels ils souscrivent. Les filtres posent des conditions à satisfaire par un message pour que l'abonné le reçoive. Les canaux sont gérés sous privilège d'utilisateur du système de publication/souscription de niveau supérieur à qui le système permet de créer et détruire les canaux.

1.2 Application éoliennes et réseau de station météorologiques

Pour explorer le potentiel du paradigme de publication/souscription ainsi que tester l'implantation que vous allez réaliser, il faudra développer une petite application liant les observations de stations et de bureaux de prévisions météorologiques à des éoliennes. L'idée générale est d'abord que les éoliennes puissent recevoir des données sur la puissance et la direction du vent courantes observées sur les stations météorologiques à leur proximité pour optimiser leur orientation par rapport aux vents. Sur réception des données de puissance et de direction des vents, l'éolienne fait une somme vectorielle pondérée par la distance entre elle et la station météorologique émettrice et ce, pour tous les vecteurs vent obtenus, puis s'oriente contre le vecteur résultant.

Un second type de données météorologiques, émises cette fois-ci par un bureau de prévisions météorologiques, sera les avis de tempêtes. Sur réception d'un tel avis la concernant (si elle est dans une région touchée par l'avis et que le niveau d'alerte de ces avis), l'éolienne doit se mettre à l'arrêt pour se protéger des conséquences de vents trop forts qui pourraient l'endommager.

1. *A priori*, car il reste possible d'ajouter l'identifiant de l'émetteur dans le message lui-même, mais le système de publication/souscription n'a pas accès à cette information.

creative commons ©J. Malenfant, 2026

```
public interface        MessageI
extends Serializable
{
    public interface    PropertyI extends Serializable
    {
        public String        getName();
        public Serializable  getValue();
    }

    public boolean      propertyExists(String name);
    public void         putProperty(String name, Serializable value);
    public void         removeProperty(String name) throws UnknownPropertyException;
    public Serializable getPropertyValue(String name) throws UnknownPropertyException;
    public PropertyI[]  getProperties();
    public MessageI     copy();
    public void         setPayload(Serializable payload);
    public Serializable getPayload();
    public Instant      getTimeStamp();
}
```

FIGURE 2 – Interfaces utilisées par les messages du système de publication/souscription.

2 Publication/souscription : messages et filtres

Avant de détailler les filtres, spécifions la teneur des messages transmis via le système de publication/souscription.

2.1 Messages

Un message dans le système de publication/souscription est d'abord un objet sérialisable car devant potentiellement être transmis entre machines virtuelles Java selon le protocole RMI. Dans le contexte du projet, les messages sont définis par des classes implantant l'interface `MessageI` [2] (voir la figure 2). Un message contient une information appelée charge utile (*payload*), c'est à dire un objet qui porte les données utiles pour l'application. Cette charge utile peut être ajoutée au message par `setPayload` et récupérée par `getPayload`.

Comme indiqué dans les généralités, les messages peuvent être filtrés. Pour cela, des propriétés sont attachées au message pour être directement accessibles au système de manière uniforme (ce qui serait plus complexe si on voulait filtrer sur la charge utile du message). Les propriétés sont de simples associations nom/valeur. Dans le projet, elles seront définies par des classes qui implantent l'interface `PropertyI` (déclarée ici comme interne à l'interface `MessageI`). Notre conception a choisi de créer les messages progressivement. Les propriétés seront donc ajoutées une à une en utilisant les méthodes `putProperty` et `removeProperty`. Les propriétés associées à un message sont accédées de deux façons :

1. Leurs valeurs sont récupérées par la méthode `getPropertyValue`.
2. L'ensemble des propriétés associées à un messages sont récupérées d'un bloc par la méthode `getProperties`.

Notre conception ajoute une estampille de temps aux messages sous la forme d'une instance de la classe standard `Instant` de Java. Cette estampille est récupérée via la méthode `getTimeStamp`.

Par ailleurs, pour faciliter l'envoi de plusieurs messages ayant les mêmes propriétés, une méthode `copy` permet de copier un message puis de modifier ses éléments, autorisant une forme de création différentielle.

2.2 Filtres

Le principe général de conception des filtres consiste à les représenter par des objets possédant une méthode `match` qui retourne vrai si le message est accepté et faux sinon. L'organisation suit cependant celle des messages, avec des filtres sur les messages qui se décomposent en filtres

2. Les interfaces présentées dans ce cahier des charges vous seront fournies comme code Java documenté en tant que point de départ du projet.

creative commons ©J. Malenfant, 2026

```
public interface        MessageFilterI
extends Serializable
{
    public interface    ValueFilterI extends Serializable
    {
        public boolean  match(Serializable value);
    }

    public interface    PropertyFilterI extends Serializable
    {
        public String       getName();
        public ValueFilterI getValueFilter();
        public boolean      match(PropertyI property);
    }

    public interface    MultiValuesFilterI extends Serializable
    {
        public String[] getNames();
        public boolean  match(Serializable... values);
    }

    public interface    PropertiesFilterI extends Serializable
    {
        public MultiValuesFilterI getMultiValuesFilter();
        public boolean            match(PropertyI... properties);
    }

    public interface    TimeFilterI extends Serializable
    {
        public boolean  match(Instant timestamp);
    }

    public PropertyFilterI[]   getPropertyFilters();
    public PropertiesFilterI[] getPropertiesFilters();
    public TimeFilterI         getTimeFilter();
    public boolean             match(MessageI message);
}
```

FIGURE 3 – Interfaces utilisées par les messages du système de publication/souscription.

sur l'estampille de temps, sur les propriétés prises individuellement et sur les propriétés prises conjointement s'il y a des contraintes croisées entre leurs valeurs. Un filtre de message (interface `MessageFilterI`, figure 3) contient donc des filtres particuliers pour les parties d'un message :

- L'interface `TimeFilterI` permet de créer des filtres sur l'estampille de temps associée à un message.
- L'interface `PropertyFilterI` permet de créer des filtres sur des propriétés individuelle ; elle s'appuie sur l'interface `ValueFilterI` qui permet de créer des filtres sur la valeur d'une propriété.
- L'interface `MultiValuesFilterI` permet de créer des filtres s'appliquant conjointement sur un ensemble de propriétés d'un message, ce qui permet d'exprimer des contraintes croisées entre les valeurs de ces propriétés.

Un filtre sur les messages peut contenir les trois types de filtres précédents *i.e.*, un filtre sur l'estampille de temps mais un ou plusieurs filtres sur les propriétés individuelles ou sur plusieurs propriétés prises conjointement. Un filtre de message peut aussi ne pas avoir de filtres de l'un ou l'autre de ces types.

Vous devriez reconnaître ici une instance d'un des patrons de conception Commande ou Interpréteur. Effectivement, les interfaces vont mener à l'implantation de classes représentant l'arborescence des parties du filtre, comme des éléments de grammaire d'un langage, et la méthode `match` est celle qui exécute ou évalue les filtres.

2.2.1 Filtres sur les propriétés individuelles

Les valeurs des propriétés sont des valeurs très générales de type `Serializable`, ce qui est conceptuellement aussi général que le type `Object` en Java. Les filtres sur les valeurs de propriétés sont définis par des classes implantant l'interface `ValueFilterI` dont la méthode `match` prend en paramètre une valeur d'une propriété dans un message. Trois principaux types de filtres sont

creative commons ©J. Malenfant, 2026

utilisables :

1. Des filtres n'acceptant qu'une seule valeur, par exemple la valeur 0 pour une propriété à valeur entière.
2. Des filtres portant sur des valeurs comparables (*i.e.*, dont la classe implante l'interface `Comparable` de Java) et qui imposent des contraintes de comparaisons sur les valeurs individuelles.
3. Un filtre *joker* qui accepte n'importe quelle valeur, ce qui sera utile pour définir des filtres qui exigent la présence d'une propriété sans contraindre sa valeur.

Un filtre sur une propriété individuelle est défini par une classe implantant l'interface `PropertyFilterI` dont la méthode `match` prend en paramètre une propriété (une instance d'une classe implantant `PropertyI`). Ces filtres précisent d'abord le nom de la propriété qu'ils ciblent, puis ils ont un filtre de valeur qui contraint la valeur de la propriété. Il est possible d'ajouter des filtres de valeur au besoin pour les applications.

2.2.2 Filtres sur plusieurs propriétés prises conjointement

Au-delà des contraintes sur les valeurs individuelles, il est possible de poser des contraintes entre les valeurs de différentes propriétés associées à un message. Par exemple, si un message contient des informations sur des personnes, on pourrait y trouver deux propriétés, taille et poids, et vouloir ne recevoir que les messages concernant les personnes jugées en obésité selon leur indice de masse corporelle (pour mémoire, IMC = poids / taille^2) *i.e.*, les personnes dont l'indice est supérieur ou égal à 30, par exemple (seuil d'obésité modérée selon l'OMC).

De tels filtres sont représentés par des classes implantant l'interface `PropertiesFilterI` dont la méthode `match` prend en paramètre un groupe (tableau) de propriétés (objets dont les classes implantent `PropertyI`). Cette interface permet de récupérer le tableau des noms de propriétés qui sont contraintes par le filtre. Elle permet aussi de récupérer le filtre sur les valeurs qui est défini par une classe implantant l'interface `MultiValuesFilterI` dont la méthode `match` prend en paramètre les valeurs des propriétés et applique la contrainte voulue entre ces valeurs. Par exemple, un filtre sur l'IMC serait défini par une classe dont les noms de propriétés concernées seraient `"poids"` et `"taille"` et le filtre sur les valeurs prendraient la valeur du poids `p` et celle de la taille `t` sous la forme d'un tableau à deux entrées pour calculer `p/(t*t) >= 30.0`.

2.2.3 Filtres sur l'estampille de temps

Un filtre sur l'estampille de temps est créé par une classe implantant l'interface `TimeFilterI` qui n'a qu'une seule méthode, la méthode `match` prenant en paramètre l'estampille de temps du message (de type `Instant`). Trois principaux types de filtres sont utilisables :

1. Les filtres n'acceptant que les estampilles plus grandes ou égales à un certain instant.
2. Les filtres n'acceptant que les estampilles plus petites ou égales à un certain instant.
3. Les filtres n'acceptant que les estampilles dans un intervalle entre deux instants.

Comme pour les filtres sur les valeurs, un filtre *joker* peut aussi être implanté à la fois pour permettre de représenter explicitement le fait que l'estampille de temps n'est pas contrainte ou pour faciliter le traitement du cas où le filtre de message ne définit pas de filtre sur l'estampille de temps, ce qui sera interprété comme ne pas la contraindre.

3 Publication/souscription : courtiers et clients

3.1 Utilisation du système par les clients

Les interfaces de composant utilisées par les clients du système de publication/souscription apparaissent à la figure 4. L'interface de composant `RegistrationCI` permet à l'utilisateur de se faire connaître du système et de gérer ses abonnements. L'interface `PublishingCI` lui sert à publier des messages alors que l'interface `ReceivingCI` sert à les recevoir. Un composant client doit prévoir des connexions sur les trois interfaces. Il requiert les interfaces `RegistrationCI` et

creative commons ©J. Malenfant, 2026

```
public interface    RegistrationCI extends OfferedCI, RequiredCI
{
    public static interface    RegistrationClassI { }
    public static enum         RegistrationClass implements RegistrationClassI
    {
        /** free access to the system.                                          */
        FREE,
        /** standard registration with a low fee.                               */
        STANDARD,
        /** high performance access registration with a higher fee.             */
        PREMIUM
    }

    public boolean  registered(String receptionPortURI) throws Exception;
    public boolean  registered(String receptionPortURI, RegistrationClass rc) throws Exception;
    public String   register(String receptionPortURI, RegistrationClass rc) throws Exception;
    public String   modifyServiceClass(String receptionPortURI, RegistrationClass rc)
                    throws Exception;
    public void     unregister(String receptionPortURI) throws Exception;

    public boolean  channelExist(String channel) throws Exception;
    public boolean  channelAuthorised(String receptionPortURI, String channel) throws Exception;
    public boolean  subscribed(String receptionPortURI, String channel) throws Exception;
    public void     subscribe(String receptionPortURI, String channel, MessageFilterI filter)
                    throws Exception;
    public void     unsubscribe(String receptionPortURI, String channel) throws Exception;
    public boolean  modifyFilter(String receptionPortURI, String channel, MessageFilterI filter)
                    throws Exception;
}

public interface    PublishingCI extends OfferedCI, RequiredCI
{
    public void     publish(String receptionPortURI, String channel, MessageI message)
                    throws Exception;
    public void     publish(String receptionPortURI, String channel, ArrayList<MessageI> messages)
                    throws Exception;
}

public interface    ReceivingCI extends OfferedCI, RequiredCI
{
    public void     receive(String channel, MessageI message) throws Exception;
    public void     receive(String channel, MessageI[] messages) throws Exception;
}
```

FIGURE 4 – Interfaces de composant utilisée par les clients du système de publication/souscription.

`PublishingCI` et offre l'interface `ReceivingCI`. Symétriquement, le composant courtier offre les interfaces `RegistrationCI` et `PublishingCI` et requiert l'interface `ReceivingCI`.

Chronologiquement, un composant client commence par s'enregistrer auprès du courtier en appelant la méthode `register` qui prend en paramètre l'URI de son port entrant offrant l'interface `ReceivingCI` et la classe de service requise parmi les classes déclarées par l'énumération `RegistrationClass`. L'URI fournie va servir d'identifiant unique pour le composant auprès du courtier. Si cette URI est déjà utilisée une exception `AlreadyRegisteredException` est lancée. [3] La première méthode `registered` permet de vérifier si un composant a déjà été enregistré sous une certaine URI. La seconde ajoute la classe de service pour laquelle il a été enregistré, mais elle lance l'exception `UnknownClientException` si le composant n'est pas déjà enregistré, peu importe sa classe de service.

La méthode `unregister` permet de se désenregistrer (ou résilier son enregistrement, si vous préférez) ; elle lance l'exception `UnknownClientException` si le composant n'est pas déjà enregistré. La méthode `modifyServiceClass` permet de modifier dynamiquement la classe de service

3. Notez que la signature de `register` dans la figure 4, comme toutes les autres d'ailleurs, ne déclare lancer que `Exception`, qui est l'exception la plus générale dont la déclaration est obligatoire. Comme expliqué en cours et en TME, pour l'instant BCM4Java n'aborde pas systématiquement la gestion des exceptions dans le contexte des composants, ce qui nous amène à utiliser cette clause `throws` « fourre-tout », permettant de lancer toutes les exceptions, jusqu'à une mise à jour globale de cette question dans le modèle BCM et son implantation en Java. Cela n'empêche pas toutefois une gestion correcte des exceptions lancées car la clause `catch` récupérant les exceptions lancées par un bout de code permet de filtrer sur des ecetpions plus précises un appel à une méthode qui se borne à déclarer qu'elle lance `Exception`.

creative commons ©J. Malenfant, 2026

d'un client ; elle retourne une nouvelle URI de port entrant offrant l'interface `PublishingCI` (ou `PrivilegedClientCI`, voir plus loin) correspondant à cette nouvelle classe mais elle lance l'exception `UnknownClientException` si le composant n'est pas déjà enregistré ou `AlreadyRegisteredException` si le composant est déjà enregistré sous la même classe de service.

L'objectif des clients est bien sûr de publier des messages sur des canaux et de recevoir des messages en s'abonnant à des canaux. Pour les clients de la classe de service `RegistrationClass.FREE`, le système propose des canaux spécifiques préexistants. Les paramètres de configuration du courtier indique le nombre N de ces canaux à créer et ceux-ci sont appelés `channel0` à `channel`$\{N-1\}$.

Les méthodes `subscribed`, `subscribe` et `unsubscribe` permettent de gérer les abonnements du composant aux canaux. La méthode `subscribe` prend en paramètres l'URI de son port entrant offrant l'interface `ReceivingCI`, le nom du canal auquel il souhaite s'abonner et un filtre à appliquer sur les messages pour que le courtier ne lui envoie que ceux qui sont acceptés par ce filtre. Cette méthode peut lancer les trois exceptions suivantes :

— `UnknownClientException` si le composant n'est pas déjà enregistré ;
— `UnknownChannelException` si le canal n'existe pas ;
— `UnauthorisedClientException` si le canal existe mais que ce composant n'est pas autorisé à l'utiliser (voir plus loin la création des canaux pour plus de détails).

Un composant peut vérifier s'il est enregistré avec les méthodes décrites précédemment. Il peut aussi vérifier l'existence d'un canal par la méthode `channelExist` et aussi s'il a l'autorisation de l'utiliser avec la méthode `channelAuthorised`. Pour plus de dynamicité, la méthode `modifyFilter` permet de changer en cours d'exécution le filtre associé à un canal auquel le composant est abonné. Cette méthode peut lancer les mêmes exceptions que `subscribe` mais en plus elle peut lancer l'exception `NotSubscribedChannelException` lorsque le composant n'est pas abonné au canal. Encore une fois, le composant peut vérifier s'il est abonné à un canal par la méthode `subscribed`. Enfin, il peut se désabonner d'un canal en appelant la méthode `unsubscribe` qui peut lancer les mêmes exceptions que `subscribe` ainsi que `NotSubscribedChannelException`.

Lors de l'enregistrement, la méthode `register` retourne une URI d'un port entrant du courtier offrant l'interface de publication correspondant à la classe de service demandée (`PublishingCI` pour les composants enregistrés dans la classe de service `RegistrationClass.FREE` ; pour les autres, voir ci-après). Le composant peut alors se connecter à ce port puis commencer à publier des messages en utilisant les méthodes `publish` de l'interface de publication. Ces deux méthodes peuvent lancer les mêmes exceptions :

— `UnknownClientException` si le composant n'est pas déjà enregistré ;
— `UnknownChannelException` si le canal n'existe pas ;
— `UnauthorisedClientException` si le canal existe mais que ce composant n'est pas autorisé à l'utiliser (voir plus loin la création des canaux pour plus de détails).

De son côté, le courtier se connecte au composant via son port entrant offrant l'interface `ReceivingCI` pour lui transmettre des messages qui sont reçus par appel des méthodes `receive`. Le composant client doit alors implanter des méthodes pour traiter les messages reçus via son port entrant offrant l'interface `ReceivingCI`. Le composant client n'est pas autorisé à lancer des exceptions lorsqu'il implante les méthodes qui vont être appelées (la clause `throws` ici est imposée par BCM4Java).

3.2 Clients privilégiés

Non seulement les classes de service standard (`RegistrationClass.STANDARD`) et *premium* (`RegistrationClass.PREMIUM`) donnent-elles accès à de meilleures performances, les clients privilégiés ont aussi des droits supplémentaires pour gérer les canaux. Pour les clients de base (classe de service `RegistrationClass.FREE`), le système ne les autorisent pas à créer ou détruire des canaux. Pour ceux-ci, le système propose des canaux spécifiques déjà créés, comme indiqué plus haut. Pour les clients des classes de service supérieures, l'URI retournée par la méthode `register` permet de se connecter à un port entrant offrant l'interface de composant `PrivilegedClientCI` (figure 5).

Cette interface de composant étend l'interface `PublishingCI` avec des méthodes qui permettent de gérer les canaux et les droits d'accès à ces derniers. La méthode `createChannel` permet ainsi de créer un canal en spécifiant son nom et une expressions régulière qui devra apparier les URIs

creative commons ©J. Malenfant, 2026

```
public interface    PrivilegedClientCI
extends PublishingCI
{
    public boolean  hasCreatedChannel(String receptionPortURI, String channel)
                    throws Exception;
    public boolean  channelQuotaReached(String receptionPortURI)
    throws Exception;
    public void     createChannel(
        String receptionPortURI,
        String channel,
        String autorisedUsers
        ) throws Exception;
    public boolean  isAuthorisedUser(String channel, String uri) throws Exception;
    public void     modifyAuthorisedUsers(
        String receptionPortURI,
        String channel,
        String autorisedUsers
        ) throws Exception;
    public void     destroyChannel(String receptionPortURI, String channel)
                    throws Exception;
    public void     destroyChannelNow(String receptionPortURI, String channel)
                    throws Exception;
}
```

FIGURE 5 – Interfaces de composant utilisée par les clients privilégiés du système de publication/souscription.

des composants admis à utiliser celui-ci. Cette méthode peut lancer les exceptions :

- `UnknownClientException` si le créateur n'est pas enregistré ;
- `AlreadyExistingChannelException` si un canal du nom fourni existe déjà ;
- `ChannelQuotaExceededException` si le créateur excède son quota de canaux qu'il peut gérer à la fois.

Les clients privilégiés sont toutefois limités dans le nombre de canaux qu'ils peuvent gérer (les limites pour chaque classe de service sont définies à la configuration du courtier). Il est possible de vérifier si un canal existe déjà par la méthode `RegstrationCI::channelExist`, si un composant est le créateur d'un canal par la méthode `hasCreatedChannel`, si un composant a atteint son quota de canaux par la méthode `channelQuotaReached` et si un composant est autorisé à utiliser un canal par la méthode `isAuthorisedUser`. Le créateur d'un canal peut ensuite modifier l'expression régulière des autorisations accordées sur un canal par la méthode `modifyAuthorisedUsers`.

Lorsqu'un canal est détruit, se pose la question de la sémantique de la destruction par rapport aux messages qui sont en transit dans ce canal. Deux méthodes sont proposées :

1. La méthode `destroyChannel` arrête immédiatement la publication des messages sur le canal (les publieurs recevront l'exception `UnknownChannelException`) puis termine l'envoi des messages encore dans la file d'attente avant de détruire le canal.
2. La méthode `destroyChannelNow` arrête immédiatement la publication et détruit le canal y compris tous les messages en file d'attente non encore expédiés.

Dans les deux cas, les ports entrant offrant l'interface `ReceivingCI` sont déconnectés.

3.3 Le courtier

Le courtier est l'élément central du système de publication/souscription. Son rôle est de gérer les enregistrements des composants clients, les canaux et les transmissions de messages en appliquant les filtres fournis par les composants récepteurs. En tant que composant, il offre les interfaces `RegistrationCI`, `PublishingCI` et `PrivilegedClientCI`. Dans un premier temps, le courtier sera réalisé par un seul composant. Dans les étapes subséquentes du projet, le courtier sera réalisé par un ensemble de composants qui vont coopérer pour assurer le rôle de courtier en commun.

3.3.1 Enregistrement

Pour s'enregistrer, les clients doivent d'abord connaître l'URI du port entrant du composant courtier offrant l'interface `RegistrationCI`. Comme indiqué dans le cours, la découverte de tels

services repose toujours en pratique sur une information qui peut être obtenue avant même de se connecter au système. Dans les applications Web, ce sont les URL qui sont ainsi connues avant l'utilisation des sites et accédées par les clients pour s'y connecter. Ici, nous allons utiliser par une approche simple qui passera aussi aux étapes suivantes avec répartition du courtier et qui consiste à fournir dans la classe de courtier une méthode statique `registrationPortURI` retournant l'URI requise.

Une fois son port sortant (requérant l'interface `RegistrationCI`) connecté, le composant client pourra s'enregistrer et s'abonner à des canaux. Avec l'URI reçue de l'appel à `register`, il pourra connecter son port sortant requérant l'interface `PublishingCI` (pour les clients de la classe de service `RegistrationClass.FREE`) ou `PrivilegedClientCI` (pour les clients des classes de service `RegistrationClass.STANDARD` ou `RegistrationClass.PREMIUM`) au port entrant du courtier pour commencer à publier des messages sur les canaux prédéfinis ou à créer des canaux puis publier sur ces derniers.

3.3.2 Gestion interne

La classe décrivant le composant courtier doit implanter l'ensemble des fonctionnalités prévues : gestion de l'enregistrement des composants clients, gestion de la création et destruction des canaux, gestion des abonnements des clients aux canaux et transmissions des messages utilisant les filtres fournis lors des abonnements. Le cahier des charges laisse une grande liberté pour les choix d'implantation, mais la qualité de ces choix et leur performance sera prise en compte dans l'évaluation.

Notez que lors de la publication des messages, plusieurs messages peuvent être envoyés à la fois par un client. Symétriquement, le courtier peut aussi livrer les messages de manière groupée. Cependant, les sémantiques de deux opérations ne sont pas liées. Certes, les messages publiés en groupe par un client peuvent être livrés en groupe, mais côté courtier, l'implantation peut choisir pour des raisons de performance de grouper plusieurs messages devant être livrés à un même client sans que ceux-ci aient été publiés en groupe.

Au fil de l'avancement du projet, d'autres activités internes au courtier vont s'ajouter, comme le nettoyage des canaux des messages devenus obsolètes, la transmission des messages entre instances de courtier dans le cas réparti, la gestion des connexions entre instances de courtier, etc.

3.4 Application

Comme indiqué en introduction, une petite application va servir à tester l'implantation du système de publication/souscription : une application liant des stations et bureaux météorologiques émetteurs de messages à des éoliennes les consommant. Trois types de composants sont donc nécessaires :

1. Le composants stations météorologiques qui vont émettre des messages sur leurs observations de la vitesse et de la direction du vent.
2. Les composants bureaux météorologiques émettant des bulletins d'alerte.
3. Les composants éoliennes qui reçoivent les messages et prennent les décisions appropriées (orientation de l'éolienne ou sa mise en sécurité).

3.4.1 Prise en compte des vents

Les stations météorologiques et les éoliennes connaissent leur position. Elles sont définies par l'interface `PositionI` (voir la figure 6) et peuvent être simplement implantées comme des points dans un espace en deux dimensions avec des coordonnées x et y. [4] L'espace est subdivisé en régions qui dont définies par l'interface `RegionI` dont la méthode `in` prend une position en paramètre et retourne vrai si cette position est dans la région et faux sinon. Les données sur le vent sont

4. Les positions peuvent aussi être implantées en coordonnées de longitude et latitude, mais il serait plus complexe de calculer des distances, à moins d'utiliser une bibliothèque comme GeographicCoordinate (`https://github.com/kloverde/java-GeographicCoordinate`). Toutefois, cela complique aussi le test de `RegionI::in`, donc cette complexité n'est pas exigée pour le projet.

creative commons ©J. Malenfant, 2026

```
public interface        PositionI
extends Serializable
{
    public boolean      equals(PositionI p);
}

public interface        RegionI
extends Serializable
{
    public boolean      in(PositionI p);
}

public interface        WindDataI
extends Serializable
{
    public PositionI    getPosition();
    public double       xComponent();
    public double       yComponent();
    public double       force();
}

public interface        MeteoAlertI
extends Serializable
{
    public interface    LevelI { }

    public static enum  Level
    {
        GREEN, YELLOW, ORANGE, RED, SCARLET
    }

    public interface    AlertTypeI { }

    public static enum  AlterType implements AlertTypeI
    {
        STORM, ICY_STORM, SNOW_STORM, FLOODING, EARTHQUAKE, ERUPTION
    }

    public AlertTypeI   getAlertType();
    public LevelI       getLevel();
    public RegionI[]    getRegions();
    public Instant      getStartTime();
    public Duration     getDuration();
}
```

FIGURE 6 – Interfaces utilisées par l'application météo/éoliennes.

représentées par des objets dont la classe implante l'interface `WindDataI`. Ces données contiennent la position d'observation (celle de la station météorologique émettrice), le vecteur vent décomposé en directions orthogonales x et y, ainsi que la force du vent qui est la longueur du vecteur précédent.

il faut un filtre sur la region et le temps

L'éolienne récupère régulièrement les données de vent en sélectionnant celles provenant des stations météorologiques dans sa proximité et qui sont également dans un intervalle de temps récent pour avoir un impact sur l'éolienne dans un délai assez court (les paramètres précis pourront être ajustés pour les besoins des tests). Le calcul à faire sur ces données pour décider de l'orientation de l'éolienne peut être assez complexe. Pour ne pas surcharger le projet, nous proposons de faire simplement une somme vectorielle pondérée par la distance entre la station émettrice et l'éolienne, puis de prendre la négation du vecteur résultant comme direction à adopter pour l'éolienne.

3.4.2 Prise en compte des alertes météorologiques

Les bureaux météorologiques émettent des alertes météorologiques sous la forme de bulletins *i.e.*, des objets dont la classe implante l'interface `MeteoAlertI`. Une alerte a un type, défini par l'énumération `AlertType`, et un niveau, défini par l'énumération `Level`. L'alerte elle-même permet de récupérer le type d'alerte par la méthode `getAlertType`, le niveau par la méthode `getLevel` et les régions concernées par la méthode `getRegions`. Elle permet aussi de récupérer l'instant du début de l'alerte par la méthode `getStartTime` et sa durée par la méthode `getDuration`. Notez que les fins d'alertes peuvent également être signalées par des alertes de niveau `Level.GREEN`

creative commons ©J. Malenfant, 2026

et potentiellement `Level.YELLOW`, selon le seuil de réaction que s'impose chaque éolienne. Les éoliennes reçoivent les alertes qui les concernent et décident de se mettre en sécurité ou de revenir au fonctionnement normal en fonction de ces dernières et de leur seuil de réaction.

3.5 Introduction de greffons

Le système de publication/souscription est destiné à être utilisé par toutes sortes de composants clients qui vont devoir s'y connecter, publier et recevoir des messages. Pour cela, il vont devoir créer des ports, les publier , les connecter, les utiliser puis les déconnecter et les dépublier, etc. Tout ce code de gestion des interactions avec le système doit être répété dans chacun des types de composants clients. Comme vu en cours, le mécanisme de greffons (*plug-ins*) de BCM4Java permet de factoriser le code répétitif dans des classes d'objets délégués spécifiques, les greffons.

Un greffon est associé à un rôle que l'on veut greffer à un composant. Pour le système de publication/souscription, plusieurs rôles sont distingués permettant d'installer les greffons selon les besoins de chaque composant client :

Enregistrement : tous les composants clients vont devoir s'enregistrer ; les signatures nécessaires pour gérer l'enregistrement font partie de l'interface de composants `RegistrationCI`.

Publication : certains composants clients vont publier des messages en utilisant les signatures de l'interface de composants `PublishingCI`.

Souscription : certains composants clients vont s'abonner à des canaux, recevoir des messages transitant par ces derniers puis s'en désabonner ; les signatures nécessaires pour l'abonnement font partie de l'interface de composants `RegistrationCI` alors que celles nécessaires pour recevoir les messages sont dans l'interface de composants `ReceivingCI`.

Gestion des canaux : les composants clients privilégiés vont créer des canaux, autorisés d'autres clients à les utiliser puis les détruire ; les signatures nécessaires pour gérer les canaux se trouvent dans l'interface de composants `PrivilegedClientCI`.

Dans un premier temps, des greffons pour les trois premiers rôles sont implantés, puis le greffon pour les clients privilégiés sera ajouté de même que de nouvelles fonctionnalités avancées pour les clients recevant des messages comme nous le décrivons plus loin.

3.5.1 Enregistrement, publication et souscription de base

Pour guider l'implantation des greffons, des interfaces Java spécifiques sont proposées en conception. La figure 7 présente les interfaces Java que les greffons correspondant aux trois premiers rôles devront implanter (au sens du « `implements` » de Java).

L'interface `ClientRegistrationI` est destinée à être implantée par le greffon d'enregistrement. Elle reprend les signatures de `RegistrationCI` permettant de gérer l'enregistrement : `registered`, `register`, `modifyServiceClass` et `unregister`. Notez que le paramètre `receptionPortURI`, présent dans les signatures de `RegistrationCI`, disparaît des signatures de `ClientRegistrationI` puisque le greffon connaîtra cette URI car c'est lui qui crée le port sortant pour se connecter au courtier. Il en sera de même pour les signatures dans les autres greffons. Il faut aussi comprendre que les signatures de `ClientRegistrationI` sont en fait utilisées dans le code du composant client pour appeler les services du courtier, donc ces appels vont passer d'abord par le greffon qui appelle à son tour le courtier via ses ports sortants.

Notez également que les exceptions lancées par les différentes signatures sont plus explicites dans `ClientRegistrationI` que dans `RegistrationCI`, ceci parce qu'une interface Java n'a pas les contraintes que les interfaces de composants subissent, ce qui simplifie le fait de rendre les exceptions lancées totalement explicites. Les explications précédentes ainsi que la documentation javadoc fournie avec les interfaces comme point de départ du projet explique clairement les raisons qui poussent à lancer les exceptions dans chaque cas.

La seconde interface de la figure 7, `ClientPublicationI` est destinée à être implantée par le greffon pour le rôle de publication. Elle reprend les signatures de l'interface de composants `PublishingCI` plus les deux signatures de l'interface de composants `RegistrationCI` permettant de s'assurer qu'un canal existe et que le composant est autorisé à l'utiliser : `channelExist` et

creative commons ©J. Malenfant, 2026

```
public interface    ClientRegistrationI extends PluginI
{
    public boolean  registered();
    public boolean  registered(RegistrationClass rc) throws UnknownClientException;
    public void     register(RegistrationClass rc) throws AlreadyRegisteredException;
    public void     modifyServiceClass(RegistrationClass rc)
                    throws UnknownClientException, AlreadyRegisteredException;
    public void     unregister() throws UnknownClientException;
}

public interface    ClientPublicationI extends PluginI
{
    public boolean  channelExist(String channel);
    public boolean  channelAuthorised(String channel)
                    throws UnknownClientException, UnknownChannelException;
    public void     publish(String channel, MessageI message)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
    public void     publish(String channel, ArrayList<MessageI> messages)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
}

public interface    ClientSubscriptionI extends PluginI
{
    // Signatures that are called by the owner component

    public boolean  channelExist(String channel);
    public boolean  channelAuthorised(String channel)
                    throws UnknownClientException, UnknownChannelException;
    public boolean  subscribed(String channel)
                    throws UnknownClientException, UnknownChannelException;
    public void     subscribe(String channel, MessageFilterI filter)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
    public void     unsubscribe(String channel)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException, NotSubscribedChannelException;
    public void     modifyFilter(String channel, MessageFilterI filter)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException, NotSubscribedChannelException;

    // Signatures that are called by the broker

    public void     receive(String channel, MessageI message);
    public void     receive(String channel, MessageI[] messages);

    // Signatures that are called by the owner component and that offer it
    // more publish/subscribe operations but implemented locally

    public MessageI waitForNextMessage(String channel)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException, NotSubscribedChannelException;
    public MessageI waitForNextMessage(String channel, Duration d)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException, NotSubscribedChannelException;
    public Future<MessageI> getNextMessage(String channel)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException, NotSubscribedChannelException;
}
```

FIGURE 7 – Interfaces utilisées par les greffons du système de publication/souscription.

`channelAuthorised`. Le fait ques les interfaces proposées par les greffons ne concordent pas exactement avec les interfaces de composants implique qu'il y aura du partage entre les greffons, ce qui devra être géré correctement.

La troisième interface, `ClientSubscriptionI`est destinée à être implantée par le greffon du rôle de souscription. Elle regroupe les signatures nécessaires pour gérer les abonnements aux canaux et recevoir des messages. Ces signatures sont subdivisées en trois groupes. Le premier groupe comporte les signatures permettant de gérer les abonnements provenant de l'interface de composants `RegistrationCI`. Comme déjà noté, ces signatures du premier groupe sont simplifiées pour

creative commons ©J. Malenfant, 2026

```
public interface    PrivilegedClientI extends PluginI
{
    public boolean  hasCreatedChannel(String channel)
                    throws UnknownClientException, UnknownChannelException;
    public boolean  channelQuotaReached() throws UnknownClientException;
    public void     createChannel(String channel, String autorisedUsers)
                    throws UnknownClientException, AlreadyExistingChannelException,
                           ChannelQuotaExceededException;
    public boolean  isAuthorisedUser(String channel, String uri)
                    throws UnknownClientException, UnknownChannelException;
    public void     modifyAuthorisedUser(String channel, String autorisedUsers)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
    public void     destroyChannel(String channel)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
    public void     destroyChannelNow(String channel)
                    throws UnknownClientException, UnknownChannelException,
                           UnauthorisedClientException;
}
```

FIGURE 8 – Interfaces utilisées par le greffon pour les clients privilégiés du système de publication/souscription.

tenir compte du fait que l'URI du port entrant servant d'identifiant sera connu du greffon. Les signatures du second groupe permettent de recevoir les messages qui proviennent du courtier via l'interface de composants `ReceivingCI`, signatures qui sont donc reprises *in extenso*.

Enfin, les signatures du troisième groupe sont plus complexes et sont expliquées dans la sous-section 3.5.3.

3.5.2 Clients privilégies

Pour les clients privilégiés, ils ont la possibilité de gérer la création, l'accès et la destruction des canaux. Les signatures de l'interface `PrivilegedClientI` de la figure 8 reprennent celles de l'interface de composants `PrivilegedClientCI` en les simplifiant de l'URI du port entrant du composant et en explicitant les exceptions lancées par chacune.

3.5.3 Réception avancée des messages

Les trois dernières signatures de l'interface `ClientSubscriptionI` de la figure 7 offrent des fonctionnalités supplémentaires pour la réception des messages par les clients *i.e.*, qui ne sont pas offertes par le courtier à travers ses propres interfaces. L'implantation de ces trois signatures doit donc être réalisée dans le greffon du rôle de souscription et fait appel aux notions de programmation parallèle et concurrente vues dans les cours 4 à 6 essentiellement.

Avant d'expliquer ces trois nouvelles signatures, rappelons comment va fonctionner la réception avec les deux signatures de l'interface de composants `ReceivingCI`. Cette interface de composants est offerte par le composant client. Lorsqu'un ou des messages doivent lui être livrés, le courtier appelle l'une ou l'autre de ces signatures pour lesquelles le port entrant du composant client devra lui-même appelé une méthode (dans le composant puis dans le greffon de souscription) qui va être chargée de réceptionner le message et de faire en sorte qu'il soit traité par le client. Le client est alors totalement passif, ne « réagissant » que lorsqu'un message lui est transmis par cette voie.

Le greffon de souscription va ajouter des manières plus actives de récupérer les messages. La première signature `waitForNextMessage` permet, dans le code du composant client, de se synchroniser sur la réception du prochain message via le canal donné. Le *thread* du composant client faisant cet appel sera bloqué jusqu'à ce qu'un message soit reçu, et lorsque le message est reçu, il le retourne en résultat. La seconde signature `waitForNextMessage` ajoute la possibilité de spécifier une durée maximale d'attente pendant laquelle le *thread* du composant client est bloqué puis, si aucun message n'est reçu à la fin de ce délai, la méthode débloque le *thread* en lui retournant `null`.

La troisième signature, `getNextMessage`, permet également de se synchroniser avec la réception

creative commons ©J. Malenfant, 2026

d'un message, mais d'une manière différente utilisant un `Future` de Java. Une variable future en programmation concurrente est une promesse qu'une valeur sera livrée mais plus tard. Lors de l'appel à `getNextMessage`, le code du composant client reçoit immédiatement ce `Future` et le *thread* peut continuer son exécution. Pour récupérer la valeur dans ce `Future`, il faut appeler la méthode `Future::get` qui, elle, retourne la valeur attendue si elle est disponible ou alors bloque le *thread* appelant jusqu'à ce que la valeur soit disponible et alors elle retourne cette valeur en débloquant le *thread* appelant. Ainsi, le composant client peut poursuivre son exécution tant qu'il n'a pas besoin de la valeur attendue puis se synchroniser sur sa réception lorsqu'il en a besoin.

Notez que rien n'empêche un composant client d'appeler ces trois méthodes plusieurs fois avant de consommer les valeurs attendues. Cela peut arriver si plusieurs *threads* en son sein font des appels aux méthodes `waitForNextMessage` ou encore si plusieurs appels sont fait à la méthode `getNextMessage` sur le *même canal*. Ainsi, ces appels « réservent » des messages qui seront reçus via ce canal dans le futur. Les demandes sont servies selon la discipline du premier arrivé, premier servi (pour les `Future`, c'est l'ordre de leur création qui compte, pas l'ordre dans lequel leurs valeurs sont accédées par la méthode `Future::get`, ce qui serait plus compliqué).

Par ailleurs, il y a évidemment une interférence notable entre les trois dernières signatures et les méthodes de réception `receive`. Que doit faire le greffon lorsqu'un message arrive, appeler `receive`, servir un `Future` ou débloquer un `waitForNextMessage` ? L'implantation devra traiter l'ensemble des attentes de messages selon la discipline du premier arrivé premier servi tant qu'il en reste en attente et n'appeler les méthodes `receive` que lorsqu'il n'y a plus aucune attente pour un message provenant d'un canal donné.

3.6 Passage à la programmation parallèle asynchrone

Considérons le scénario de ce qui se passe lorsqu'un client A publie un message qui doit être reçu par un autre client B. Les étapes sont :

1. Le composant A appelle son port sortant qui appelle le connecteur qui appelle à son tour le port entrant du courtier.
2. Le port entrant du courtier soumet une tâche au composant courtier pour propager le message.
3. La propagation du message au sein du courtier implique d'abord d'identifier les composants qui doivent recevoir le message, ce qui exige d'exécuter les étapes suivantes :
   (a) Retrouver tous les abonnées au canal de publication du message, dont B dans ce scénario.
   (b) Passer le message au filtre de chacun des abonnés, dont le filtre posé par B lors de son abonnement.
   (c) Si le filtre retourne vrai, alors il faut retrouver le port sortant de réception associé à l'abonné.
4. Pour chaque port sortant via lesquels le message doit être transmis, le courtier appelle ce port sortant avec les méthodes `receive`, port qui appelle le connecteur, qui lui-même appelle le port entrant du composant récepteur, dont celui de B dans notre scénario.
5. Le port entrant de B appelle la méthode de réception des messages de B que ce dernier va traiter.

La description précédente s'intéresse à ce qui doit être réalisé (le quoi). Intéressons-nous maintenant à la façon dont tout ceci est réalisé (le comment). Si vous avez suivi l'approche de base de BCM4Java pour implanter cette séquence de traitement, vous avez des composants avec un seul *thread* et des appels synchrones via la méthode `handleRequest`. On constate dans ce cas :

— que le *thread* prend en charge l'étape 1 puis se bloque dans l'attente de la fin d'exécution de son appel au port entrant du courtier ;
— que le *thread* du courtier prend en charge toutes les étapes de 1 à 4 puis se bloque dans l'attente de la fin d'exécution de son appel au port entrant de B ;
— que le *thread* de B prend en charge l'étape 5 puis retourne de cette exécution, ce qui déclenche en cascade le retour de l'appel du courtier à B puis le retour du courtier de l'appel de A, qui peut reprendre son exécution.

De ce point de vue des *threads* et de la synchronisation, on constate :

— que l'appel synchrone fait par A va bloquer son *thread* jusqu'à ce que le courtier ait propagé le message à tous ses récepteurs, donc jusqu'à et y compris la livraison et le traitement du message par tous ses récepteurs ;
— que la propagation au sein du courtier va mobiliser un *thread* du composant courtier de bout en bout, jusqu'à la livraison et le traitement du message par tous les récepteurs donc, s'il n'y a qu'un seul *thread* dans ce composant, il ne pourra pas propager d'autres messages tant que tous les récepteurs n'auront pas reçu et traiter le message ;
— que pour chaque livraison à un récepteur, les récepteurs suivants ne vont recevoir le message que lorsque tous les récepteurs précédents l'auront reçu et traité.

Si on procéde de cette manière naïve, ce serait non seulement un énorme gâchi de ressources mais également un risque énorme couru par les composants. En effet, si un composant récepteur comme B devait prendre un temps déraisonnable à traiter le message, il bloquerait indéfiniment A et le courtier (et empêcherait ainsi toute autre publication d'autres composants clients pendant ce temps) de même qu'il empêcherait tous les récepteurs suivants de recevoir et traiter le message !

Pour pallier ces difficultés, il faut passer à la programmation asynchrone. En programmation asynchrone, les appels de méthodes ne produisent pas de résultat ; l'appelant n'attend donc pas et poursuit son exécution immédiatement après que l'appel a été accepté (*i.e.*, la tâche devant exécuter la méthode appelée a été uniquement soumise au composant appelé, sans attendre qu'elle soit effectivement exécutée). En programmation asynchrone, si un résultat doit logiquement être renvoyé, l'appelé le transmet par un appel en retour via une autre méthode plus tard, lorsque son service a été exécuté et qu'il a produit le résultat attendu.

Pour mieux comprendre l'approche synchrone versus l'approche asynchrone, prenons une situation de la vie courante. Imaginons que vous souhaitez faire une réservation dans un hôtel (qui n'a pas de site WWW). Vous avez deux options : soit vous appelez l'hôtel, soit vous envoyez un courrier électronique. L'appel téléphonique suit l'approche synchrone. Vous êtes en communication avec l'hôtel immédiatement. L'hôtelier traite votre demande sur le champ. Et vous obtenez votre confirmation de réservation avant de raccrocher, ayant été bloqué par votre appel téléphonique jusqu'à l'obtention de cette confirmation (orale). Le courrier électronique suit l'approche asynchrone. Vous envoyez votre message de demande de réservation à l'hôtelier puis, sans attendre, vous pouvez passer à autre chose en attendant la réponse. Le message est reçu puis l'hotelier le traite lorsqu'il devient disponible. Il vous envoie ensuite un courrier électronique en retour pour vous confirmer la réservation.

Concernant le projet, le passage à la programmation parallèle asynchrone ne nécessite pas de modifications des interfaces de composants ni des interfaces Java présentées jusqu'ici car la communication par publication/souscription est par essence asynchrone. D'abord, une grande partie des méthodes vont demeurer synchrones. C'est le cas :
— de toutes les méthodes appelées via l'interface de composants `RegistrationCI` ;
— de toutes les méthodes appelées via l'interface de composants `PrivilegedClientCI`.

Par contre, les méthodes appelées via les interfaces de composants `PublishingCI` et `ReceivingCI` vont être maintenant appelées de manière asynchrone. Ceci se fait en utilisant la méthode `AbstractComponent::runTask` (et les autres méthodes similaires), comme vu en cours.

L'introduction du parallélisme concerne principalement le composant courtier qui va utiliser différents *pools* de *threads* pour exécuter ses différentes fonctions : réception des messages depuis les composants clients les publiant, propagation des messages en interne vers les canaux et les abonnements pour filtrage, livraison des messages aux composants devant les recevoir. La séparation de ces fonctions sur différents *pools* de *threads* va permettre d'arbitrer les ressources consacrées à chacune de ces fonctions selon leurs besoins afin d'obtenir la meilleure performance globale possible. Il sera aussi utilisé pour différencier les performances offertes aux clients de différentes classes de service. Enfin, il sera aussi nécessaire au sein des composants clients pour implanter et utiliser les fonctionnalités de réception avancée décrites ci-haut. Conjointement avec l'introduction du parallélisme, il faudra assurer la gestion de la concurrence en introduisant les synchronsations nécessaires à une exécution parallèle sûre.

3.7 Répartition

À venir.

4 Déroulement du projet et résultats attendu

Le déroulement du projet s'étale sur quatre étapes échelonnées autour des quatre évaluations successives, audits et soutenances.

4.1 Résumé du déroulement du projet

Dans ses grandes lignes, le projet va se dérouler sur tout le semestre de la manière suivante :

1. **Première étape** (3 semaines)
   Semaine 1 (26/01) : Implantation (en Java) des messages et des filtres (§2), incluant les tests unitaires en JUnit.
   Semaine 2 (2/02) : Implantation (en BCM4Java) des clients et du courtier centralisé, en se limitant pour l'étape 1 aux fonctionnalités destinées aux clients de la classe de service `RegisrationClass.FREE` (§3.1 et §3.3). Premiers tests avec des clients basiques qui publient et reçoivent des messages simples (juste pour tester le bon fonctionnement des appels).
   Semaine 3 (9/02)) : Implantation des clients de l'application météo/éolienne (§3.4). Construction des premiers scénarios de tests simples (pas nécessairement avec l'outil de test de l'annexe B) sur cette application.
2. **Deuxième étape** (4 semaines)
   Semaine 4 (16-17/02) : (en parallèle avec l'audit 1) Implantation des fonctionnalités offertes aux clients privilégiés (§3.2).
   Semaine 5 (23-24/02) : Passage des composants clients aux greffons (§3.5 sans §3.5.3).
   Semaine des congés d'hiver (2-3/03) : Complément de l'implantion et préparation de scénarios de test temporisés (voir annexe B) en vue du rendu de code et de la soutenance pour l'évaluation de mi-semestre.
   Semaine des premiers examens répartis (9-10/03) : soutenances de mi-semestre (l'ordre de passage sera transmis en avance).
3. **Troisième étape** (2 semaines)
   Semaine 6 (16-17/03) : Gestion partielle du parallélisme et de la concurrence au sein du courtier (§3.6) suffisante pour désynchroniser la publication d'un message, la propagation de ce dernier au sein du courtier et sa réception par l'abonné.
   Semaine 7 (23-24/03) : Implantation des fonctionnalités avancées de réception des messages dans le greffon de souscription et les composants clients (§3.5.3).
4. **Quatrième étape** (7 semaines)
   Semaine 8 (30-31/03) : (en parallèle avec l'audit 2) Extension aux courtiers répartis (§3.7).
   Semaine du congé Pascal (6-7/04) : Poursuite de l'extension aux courtiers répartis ; début de l'implantation d'une exécution répartie.
   Semaine 9 (13-14/03) : Construction d'une première exécution répartie sur deux processus Unix, chaque JVM ayant au moins un composant courtier réparti.
   Semaines des congés de printemps (20-21 et 27-27/04) : Complément de l'implantation du parallélisme au sein des courtiers pour améliorer leur performance interne et pour offrir des performances différentes en fonction de la classe de service des clients. Poursuite de l'implantation des courtiers répartis et de la réalisation de l'exécution répartie.
   Semaine 10 (4/05) : Préparation de scénarios de test en vue du rendu de code et de la soutenance pour l'évaluation de mi-semestre.
   Semaine des deuxièmes examens répartis (11-12/05) : soutenances finales (l'ordre de passage sera transmis en avance).

Ce déroulement est proposé à titre indicatif. Il permet d'assurer les objectifs attendus des évaluations successives. Toutefois, il est fortement suggéré de ne pas hésiter à avancer plus vite si vous le pouvez ; cela réduira votre charge de travail plus tard dans le semestre, lorsque vous serez plus sollicités par vos autres unités d'enseignement. Par exemple, si vous avez terminé ce qui est demandé pour l'audit 1 avant la séance de TD/TME de la semaine 4, n'hésitez pas à commencer le travail attendu pour les semaines suivantes.

creative commons ©J. Malenfant, 2026

4.2 Objectifs pour l'audit 1

La première étape est centrée sur une première implantation des fonctionnalités du système de publication/souscription pour les clients de la classe de service `RegistrationClass.FREE`, ce qui demandera une prise en main du projet suffisante pour :

- implanter les composants courtiers et clients de l'application météo/éolienne ainsi que tous les éléments BCM4Java nécessaires (ports, connecteurs, etc.) ;
- exécuter la création et l'interconnexion d'un courtier, d'au moins une éolienne, d'au moins deux stations météorologiques et d'au moins un bureau météorologique ;
- monter un scénario de test simple (sans l'outil de test temporisé) où les stations météorologiques et le bureau météorologique émettent au moins un message filtré et reçu par l'éolienne.

Pour cet audit, le principal critère d'évaluation sera le degré de réalisation des développements décrits dans la première partie du projet, leur conformité aux exigences du cahier des charges et l'exécution correcte d'un scénario de test demandé pour démontrer le bon fonctionnement de l'ensemble.

4.3 Objectifs pour la soutenance de mi-semestre

L'ensemble des étape 1 et 2 feront l'objet de l'évaluation de mi-semestre qui prendra la forme d'une soutenance avec rendu préalable de code (voir plus loin les modalités d'évaluation). Pour cette évaluation, les critères de notation porteront d'abord sur l'état d'avancement de votre code suffisante pour :

- implantater complètement les fonctionnalités des composants courtier et clients de l'application météo/éolienne décrites dans ces étapes ci-haut ;
- implanter les greffons pour les rôles d'enregistrement, de publication et de souscription ;
- implanter tous les ports et connecteurs correspondant aux interfaces de composants ajoutées ;
- monter un scénario de test temporisé à plusieurs éoliennes, plusieurs stations météo et plusieurs bureaux météo couvrant l'ensemble des types de filtres sur les messages et les fonctionnalités pour les clients des trois classes de service.

Vous devrez rendre votre code le dimanche soir précédant la soutenance (voir les modalités décrites ci-après). Pour cette étape, en plus du taux de couverture des réalisations demandées (étapes 1 et 2) et de leur bon fonctionnement en exécution démontré par les scénarios de test demandés, la qualité de l'ensemble de votre code sera également un critère d'évaluation :

- lisibilité et compréhensibilité ;
- pertinence des choix de conception ;
- qualité de la programmation Java ;
- pertinence des choix d'implantation.

4.4 Objectifs pour l'audit 2

L'objectif principal de la troisième étape sera d'ajouter l'exécution asynchrone des requêtes, assurer une gestion explicite du parallélisme par utilisation de *pools* de *threads* créés explicitement et maîtriser la concurrence par l'introduction de sections critiques dans le code.

L'audit 2 se déroulera pendant la huitième séance de TME et l'état d'avancement de votre code suffisante pour :

- exécuter des appels aynchrones pour la publication, la propagaiton et la réception des messages ;
- utiliser des *pools* de *threads* distincts pour la réception des messages, leur propagation et leur livraison ;
- la gestion de la concurrence ;
- illustrer l'utilisation des modes de réception avancés des messages par les composants clients ;
- les scénarios de tests d'intégration temporisés permettant de vérifier le bon fonctionnement des appels asynchrones, du parallélisme et de la concurrence en faisant s'exécuter plus d'une

creative commons ©J. Malenfant, 2026

requête à la fois sur les mêmes composants.

Pour cet audit également, le principal critère d'évaluation sera le degré d'avancemtn dans la réalisation des développements demandés et le bon fonctionnement des tests d'intégration.

4.5 Objectfs pour la soutenance finale

À venir.

5 Modalités générales de réalisation et calendrier des évaluations

— Le projet se fait **obligatoirement** en **équipe de deux personnes**. Tous les fichiers sources du projet doivent comporter les noms (balise `authors`) de tous les auteurs en Javadoc. Lors de sa formation, chaque équipe devra se donner un nom et me le transmettre avec les noms ert le numéro d'étudiant des personnes la formant au plus tard le **28 janvier 2026**.
— Le projet doit être réalisé avec **Java SE 8**. Attention, peu importe le système d'exploitation sur lequel vous travaillez, il faudra que votre projet s'exécute correctement sous Eclipse et sous **Mac Os X/Unix** (que j'utilise et sur lequel je devrai pouvoir refaire s'exécuter tous vos tests).
— L'évaluation comportera quatre épreuves : deux audits intermédiaires, une soutenance à mi-semestre et une soutenance finale, ces dernières accompagnées d'un rendu de code et de documentation. Ces épreuves se dérouleront selon les modalités suivantes :

1. Les deux audits intermédiaires dureront 5 à 10 minutes au maximum (par équipe) et se dérouleront lors des séances de TME. Le premier audit se tiendra pendant la séance 4 (**16 février 2026**) et il portera sur votre avancement de l'étape 1. Le second audit se déroulera lors de la séance 9 (**ATTENTION, modifié au *30 mars 2026***) et il portera sur votre avancement de l'étape 3. Ils compteront chacun pour 5% de la note finale de l'UE.
2. La **soutenance à mi-parcours** d'une durée de 30 minutes portera sur l'atteinte des objectifs de l'ensemble des première et deuxième étapes du projet. Elle se tiendra pendant la semaine des premiers examens répartis du **9 au 13 mars 2026** (très probablement les lundi 9/03 et mardi 10/03)) selon un ordre de passage et des créneaux qui seront annoncés au préalable. Elle comptera pour 35% de la note finale de l'UE. Elle comportera une discussion des réalisations devant l'écran sous Eclipse et une démonstration (exécution) des tests. Les rendus à mi-parcours se feront le **dimanche 8 mars 2026 à minuit** au plus tard (des pénalités de retard seront appliquées).
3. La **soutenance finale** d'une durée de 30 minutes portera sur l'ensemble du projet mais avec un accent sur les troisième et quatrième étapes. Elle aura lieu dans la semaine des seconds examens répartis, probablement les lundi 11/05/2026 et mardi 12/05/2026, selon un ordre de passage qui sera annoncé au préalable. Elle comptera pour 55% de la note finale de l'UE et comportera une discussion sur vos choix de conception et d'implantation ainsi que sur les réalisations suivie d'une démonstration (exécution) des tests. Les rendus finaux se feront le **dimanche 10 mai 2026 à minuit** au plus tard (des pénalités de retard seront appliquées).

— Lors des soutenances, les points suivants seront évalués :
  — le respect du cahier des charges et la qualité de votre programmation ;
  — l'exécution correcte de tests unitaires (JUnit pour les classes et les objets Java et, le cas échéant, scénarios de tests pour les composants) ;
  — l'exécution correcte de tests d'intégration mettant en œuvre des composants de tous les types ;
  — la qualité des scénarios de tests, conçus pour mettre à l'épreuve les choix d'implantation ;
  — la qualité de votre code autant dans la technicité que dans la maintenabilité (votre code doit être clair, commenté, lisible – choix pertinents des identifiants, . . . – et correctement présenté – indentation, . . . – etc.) ;
  — la qualité de la documentation (vos rendus pour la soutenance finale devront inclure une *documentation Javadoc* des paquetages et classes de votre projet générée et incluse

creative commons ©J. Malenfant, 2026

dans votre livraison dans un répertoire `doc` au même niveau que vos répertoires de code source.

— Bien que les audits et soutenances se fassent par équipe, l'évaluation reste **individuelle**. Lors des audits et des soutenances, *chaque membre* devra se montrer capable d'expliquer différentes parties du projet, et selon la qualité de ses explications et de ses réponses, sa note peut être supérieure, égale ou inférieure à celle des autres membres de son équipe.

— Lors des soutenances, **tout retard** non justifié de membres de l'équipe de plus d'un tiers de la durée de la soutenance entraînera une **absence** et une note de 0 attribuée aux membre(s) retardataire(s) pour l'ensemble de l'épreuve concernée (y compris le rendu préalable). Si une partie des membres d'une équipe arrivent à l'heure ou avec un retard de moins d'un tiers de la durée de la soutenance, les présents passeront l'épreuve sans les retardataires.

— Le rendu à mi-parcours et le rendu final se font sous la forme d'une archive `tgz` si vous travaillez sous Unix ou `zip` si vous travaillez sous Windows ***à l'exclusion de toute autre format*** (n'inclure *que les sources*, c'est à dire uniquement les répertoires de code source — `src` ou autre répertoires que vous aurez créés — de votre projet *sans les binaires* et les jars pour réduire la taille du fichier) envoyé à `Jacques.Malenfant@lip6.fr` comme attachement fait proprement avec votre programme de gestion de courrier préféré ou encore par téléchargement avec un lien envoyé par courrier électronique (en lieu et place du fichier). Donnez pour nom au répertoire de projet et à votre archive celui de votre équipe (ex. : équipe LionDeBelfort, répertoire de projet `LionDeBelfort` et archive `LionDeBelfort.tgz`).

— **Tout manquement à ces règles élémentaires entraînera une pénalité dans la note des épreuves concernées !**

— Pour la **deuxième session**, si elle s'avérait nécessaire, elle consiste à poursuivre le développement du projet pour résoudre ses insuffisances constatées à la première session et donnera lieu à un rendu du code et de documentation puis à une soutenance dont les dates seront déterminées en fonction du calendrier du master. Les critères d'évaluation sont les mêmes que lors de la soutenance finale, mais modulés selon qu'une seule personne ou deux de la même équipe doivent passer la seconde session. Sur demande faite en avance (dès les notes de première session connues), une personne peut choisir de passer la seconde session seule même si d'autres membres de l'équipe doivent également le passer ; en cas de désaccord entre les membres de l'équipe, la demande de passage en solitaire prévaudra.

creative commons ©J. Malenfant, 2026

A Scénarios de tests et horloge accélérée

Le test des applications avec interfaces graphiques et encore plus des applications parallèles et réparties exige de pouvoir planifier des actions de différentes entités logicielles à des instants précis les unes par rapport aux autres pour construire des scénarios d'exécution corrects et réalistes. Par exemple, si dans une application Web une action d'un acteur X provoque la création d'une entrée en base de données qui peut ensuite être lues par un autre acteur Y, un scénario de test correct doit s'assurer que l'action de l'acteur X soit bien exécutée avant l'action de l'acteur Y. Dans les tests séquentiels, le simple fait de programmer l'exécution de l'action de A avant celle de B suffit à assurer l'exactitude du scénario. Dans une application parallèle, les actions de deux acteurs étant réalisées dans deux *threads*, on doit les synchroniser pour assurer l'exactitude du scénario. Plusieurs techniques sont envisageables pour ce faire, dont celle d'une synchronisation *dirigée par le temps* (*time-triggered*) que nous allons utiliser dans le cadre du projet CPS.

A.1 Scénarios de test temporisés

Pour le projet, par exemple, vous comprendrez vite que lors du démarrage, il faut que les composants clients privilégiés du système de publication/souscription créent les canaux auprès du courtier (action 1) avant que d'autres clients puissent s'abonner à ces mêmes canaux (action 2). Si on ne synchronise pas ces actions, des conditions de course (*race conditions*) vont prévaloir entre les composants et selon l'ordre dans lequel la machine virtuelle Java va exécuter les choses, on pourra très bien avoir l'action 2 qui sera exécutée avant l'action 1, ce qui mènera à une erreur.

BCM4Java offre une classe appelée `AcceleratedClock` ainsi qu'un composant `ClocksServer` pour simplifier la construction de scénarios de test temporisés. Premièrement, pour synchroniser les actions des composants de manière dirigée par le temps, il faut que tous les composants puissent accéder à une unique horloge qui va établir un temps global suffisamment précis pour qu'on puisse faire exécuter une série d'actions par différents composants dans un ordre choisi par le testeur. Que l'on soit sur un seul ordinateur ou sur plusieurs ordinateurs dont les horloges matérielles sont synchronisées avec suffisamment de précision, l'horloge matérielle est une bonne candidate pour servir de temps global.

En Java, la méthode `System::currentTimeMillis` retourne la valeur de l'horloge matérielle de l'ordinateur sous-jacent exprimée en ce qui est appelé l'*Unix epoch time*, c'est à dire en temps écoulé depuis minuit le 01/01/1970 en millisecondes. Par ailleurs, les *pools* de *threads* ordonnançables de l'`ExecutorService` de Java offrent la possibilité de faire exécuter du code après un certain délai exprimé en temps réel s'appuyant sur l'*Unix epoch time* pour ordonnancer cette exécution au bon moment. BCM4Java s'appuie sur cette implantation pour offrir la méthode `scheduleTask` :

```
final AbstractComponent c = this;
this.scheduleTask(
  o -> { c.traceMessage("Ceci sera exécuté après le délai d nanosecondes.\n"); },
  d, TimeUnit.NANOSECONDS);
```

Attention, pour pouvoir utiliser la méthode `scheduleTask` (et les autres méthodes du même genre), il faut que le composant possède au moins un *thread* dans un *pool* de *threads* qui soit ***ordonnançable***.

A.2 Horloge accélérée

Bien que l'horloge matérielle donne une base fixe et fiable sur plusieurs ordinateurs, planifier des actions de test dans cette référence temporelle n'est pas aisée. Pour faciliter la planification des actions, la classe `AcceleratedClock` de BCM4Java offre la possibilité d'utiliser une autre référence temporelle pour planifier les actions : celle de la classe `Instant` de Java. Sans entrer dans tous les détails, il est facile de créer un instant « virtuel » grâce à la méthode statique `Instant#parse` qui prend en paramètre une chaîne de caractères où un instant est exprimé selon la norme internationale suivante :

creative commons ©J. Malenfant, 2026

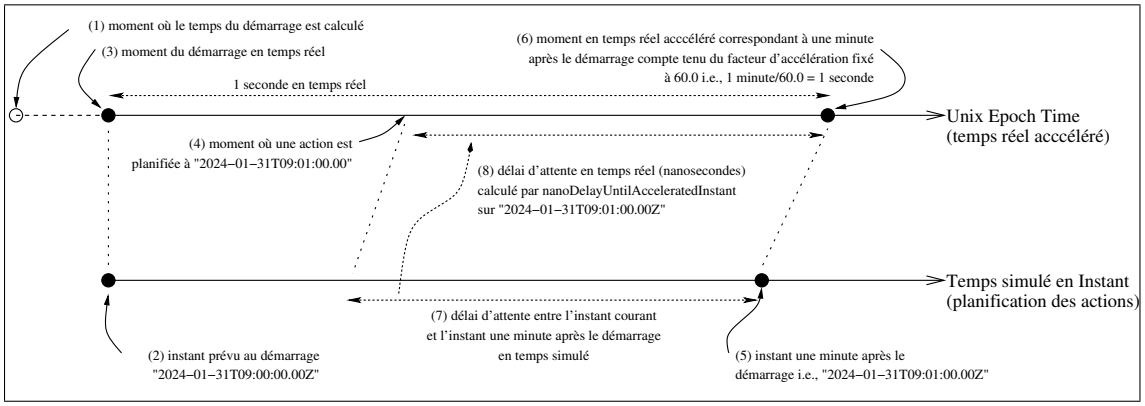


FIGURE 9 – Les deux références temporelles mises en synchronisation par `AcceleratedClock` : le temps réel accéléré s'appuyant sur l'horloge matérielle de l'ordinateur via le système d'exploitation et le temps virtuel des actions du scénario de test représenté par des `Instant` de Java.

"2025-01-31T09:00:00.00Z"

c'est à dire année-mois-jour puis heure, minutes, secondes et centièmes de secondes [5], le Z indiquant le fuseau horaire. L'expression `Instant.parse("2024-01-31T09:00:00.00Z")` retourne donc l'objet instant représentant le 31 janvier 2025 à 9 heures du matin.

Supposons que l'instant précédent représente l'instant de démarrage des actions du scénario de test, grâce aux instants Java on peut planifier une action devant se dérouler une minute après le démarrage à 9h01. Soit on crée un nouvel instant avec :

```
Instant.parse("2024-01-31T09:01:00.00Z")
```

Soit on crée de manière équivalente un nouvel instant par rapport à un instant précédent :

```
Instant i0 = Instant.parse("2025-01-31T09:00:00.00Z");
Instant i1 = i0.plusSeconds(60);
```

On constate qu'il sera beaucoup plus facile et lisible de travailler avec des instants Java pour planifier les actions d'un scénario de test qu'avec l'*Unix epoch time* en millisecondes.

Maintenant, la question qui se pose est comment mettre en relation un temps simulé exprimé en `Instant` avec le temps Unix qui sert de base pour ordonnancer l'exécution de tâches à des moments précis en temps réel. L'idée va être de créer une horloge qui va maintenir une synchronisation entre les deux références temporelles. Cette classe, `AcceleratedClock`, va permettre de créer une horloge en lui passant un moment précis en temps réel Unix pour le démarrage des actions du scénario de test disons `t0` et un instant en temps simulé du scénario exprimé en `Instant` disons `i0`. Après le démarrage du scénario (après `t0`), l'horloge va permettre de convertir des temps réel Unix (postérieurs à `t0`) en instants de même que des instants (postérieurs à `i0`) en temps réel Unix.

Un autre aspect intéressant à prendre en compte est la possibilité de planifier des actions selon une échelle de temps commode mais de pouvoir ensuite exécuter le scénario beaucoup plus rapidement. Par exemple, on pourrait trouver plus simple de planifier un scénario de test avec des actions se déclenchant toutes les minutes, mais ensuite d'accélérer ce temps pour faire en sorte qu'une minute planifiée s'exécute en une seconde en temps réel. Pour ce faire, `AcceleratedClock` va permettre de définir à la création d'une horloge un facteur d'accélération entre le déroulement du temps simulé et le déroulement du temps réel.

Pour comprendre, considérez la figure 9. En (1), on va calculer et programmer le temps de démarrage du scénario de test. Souvent, ce sera fait dès le début de l'exécution de l'application et

5. **Attention**, la norme Java n'impose pas à la classe `Instant` une précision sous la seconde. L'implantation de cette classe n'a donc pas nécessairement à fournir les centièmes de secondes. Il semble que les implantations de Java sous Windows ne sont précises qu'à la seconde alors que celles sous Unix le soit au centième de secondes, ce qui impacte les délais minimum ainsi que les facteurs d'accélération qu'on peut utiliser sans observer d'erreur dans l'exécution des scénarios de test.

creative commons ©J. Malenfant, 2026

il faudra laisser suffisamment de temps pour l'initialisation et le démarrage de tous les composants. En définissant un délai initial de démarrage, on pourrait définir le moment de démarrage de la manière suivante :

```
public static final long START_DELAY = 3000L; // 3000 millisecondes
long unixEpochStartTimeInNanos =
  TimeUnit.MILLISECONDS.toNanos(System.currentTimeMillis() + START_DELAY);
```

En (2), apparaît l'instant de départ matérialisé sur la référence temporelle du scénario de test et en (3) le moment du démarrage en temps réel est matérialisé sur la référence temporelle du temps réel Unix.

Pour pouvoir se référer à une horloge, on lui attribue à sa création une URI. Ainsi, la création d'une horloge accélérée peut se faire de la manière suivante :

```
public static final String CLOCK_URI = "horloge-101";
AcceleratedClock ac =
  new AcceleratedClock(
      CLOCK_URI,        // URI attribuée à l'horloge
      t0,               // moment du démarrage en temps réel Unix
      i0,               // instant de démarrage du scénario
      60.0)             // facteur d'acccélération
```

Avant de pouvoir faire des calculs avec cette horloge, il faut attendre le moment du démarrage en temps réel. Pour cela, la classe `AcceleratedClock` définit une méthode `waitUntilStart()` qui, comme son nom l'indique bloque le *thread* qui l'appelle jusqu'à ce que l'on arrive en temps réel au temps `t0` choisi pour le démarrage du scénario.

Supposons maintenant qu'à un certain moment dans l'exécution (voir (4) sur la figure) se situant avant le moment en temps réel où elle doit être déclenchée, le programme planifie une action devant se déclencher à l'instant `i1`, c'est à dire une minute après le démarrage du scénario de test qui est matérialisé en (5). Grâce à la synchronisation modulo le facteur d'accélération, `AcceleratedClock` peut reporter cet instant `i1` sur la référence temporelle du temps réel Unix en (6). Elle peut alors calculer le délai en termes d'instants à attendre depuis le moment où on veut planifier cette action jusqu'à `i1` en (7). Elle peut aussi calculer le même délai mais cette fois-ci en temps réel Unix, indiqué en (8). Ce délai en temps réel est celui qu'il faut utiliser dans la méthode `scheduleTask` pour faire s'exécuter l'action au bon moment :

```
long d = ac.nanoDelayUntilInstant(i1); // délai en nanosecondes
final AbstractComponent c = this;
this.scheduleTask(
  o -> { c.traceMessage("Le code de l'action 1 doit être placé ici.\n"); },
  d, TimeUnit.NANOSECONDS);
```

Le moment où la planification est faite est le moment auquel l'expression `ac.nanoDelayUntilInstant(i1)` est exécutée et le délai `d` est le délai en temps réel Unix à attendre avant de déclencher l'exécution de l'action 1.

A.3 Partage d'horloges accélérées : le serveur d'horloge

Maintenant que l'on a vu les principales fonctionnalités de l'horloge accélérée [6], se pose un nouveau problème : comment faire en sorte que plusieurs composants, s'exécutant potentiellement sur plusieurs ordinateurs, puissent partager une *même* horloge accélérée afin que leurs actions planifiées soient effectivement synchronisées selon le temps qui s'écoule sur cette horloge.

Comme on l'a répété en cours, on ne peut pas partager une référence Java à un objet horloge entre plusieurs machines virtuelles Java. La solution consiste à fournir un composant serveur d'horloge sur lequel on va créer des horloges avec leurs URIs puis les composants vont pouvoir appeler

6. Consultez la documentation javadoc de BCM4Java pour plus de détails.

ce composant pour récupérer des copies de cette horloge en utilisant l'interface `ClocksServerCI` offerte par `ClocksServer` et requise par les composants qui souhaitent l'utiliser. Toutes les copies d'une même horloge seront synchronisées entre elles par le fait qu'elles utilisent l'horloge matérielle comme référence temporelle. Que ces composants soient tous sur le même ordinateur et donc se réfèrent à la même horloge matérielle par définition offrant une synchronisation parfaite, ou qu'ils soient sur différents ordinateurs donc les horloges matérielles de ces ordinateurs seront supposées suffisamment synchronisées entre elles pour que les horloges accélérées le soient également.

Dans le cadre du projet, il suffira de créer un composant serveur d'horloge à partir de la classe `ClocksServer`. Ce composant permet de créer une horloge en passant les paramètres nécessaires à l'un de ses constructeurs. Par exemple, pour l'horloge précédente, cela donne :

```
public static final String TEST_CLOCK_URI = "test-clock";
public static final Instant START_INSTANT =
                               Instant.parse("2024-01-31T09:00:00.00Z");
protected static final long START_DELAY = 3000L;
public static final double ACCELERATION_FACTOR = 60.0;

long unixEpochStartTimeInNanos =
  TimeUnit.MILLISECONDS.toNanos(System.currentTimeMillis() + START_DELAY);

AbstractComponent.createComponent(
  ClocksServer.class.getCanonicalName(),
  new Object[]{
        TEST_CLOCK_URI,             // URI attribuée à l'horloge
        unixEpochStartTimeInNanos,  // moment du démarrage en temps réel Unix
        START_INSTANT,              // instant de démarrage du scénario
        ACCELERATION_FACTOR});      // facteur d'acccélération
```

Ensuite, chaque composant qui veut se synchroniser sur cette horloge va devoir se connecter au serveur d'horloge, récupérer une copie de l'horloge en utilisant son URI puis l'utiliser pour faire sa planification :

```
ClocksServerOutboundPort p = new ClocksServerOutboundPort(this);
p.publishPort();
this.doPortConnection(
        p.getPortURI(),
        ClocksServer.STANDARD_INBOUNDPORT_URI,
        ClocksServerConnector.class.getCanonicalName());
AcceleratedClock ac = p.getClock(TEST_CLOCK_URI);
this.doPortDisconnection(p.getPortURI());
p.unpublishPort();
p.destroyPort();
// toujours faire waitUntilStart avant d'utiliser l'horloge pour
// calculer des moments et instants
ac.waitUntilStart();
```

A.4 Précautions à prendre

La précision de l'ordonnancement sur un ordinateur utilisant un système d'exploitation comme Unix n'est pas très élevée. Par expérience sur mon Unix, la précision est d'environ 10 millisecondes en pratique. Cela veut dire que si deux composants appellent `scheduletask` au même moment avec des délais différant de moins de 10 millisecondes, on ne peut plus prévoir dans quel ordre les deux tâches seront exécutées.

En pratique, cela veut dire que le délai en temps réel séparant deux tâches à ordonnancer ne doit pas être inférieur à 10 millisecondes pour obtenir l'ordre que l'on souhaite dans un scénario. Et bien sûr, il faut prendre en compte le facteur d'accélération qui aboutit à réduire les délais en temps réel entre les actions planifiées à deux instants. Il faut donc éviter de produire des scénarios où le délai

creative commons ©J. Malenfant, 2026

entre deux actions selon la référence temporelle des instants divisé par le facteur d'accélération ne donne une valeur inférieure à 10 millisecondes. Par exemple, deux actions planifiées à une minute d'intervalle dans le temps simulé avec un facteur d'accélération de 600 vont se produire à un dixème de secondes de distance (60 secondes/600) en temps réel, donc ne devrait pas poser de problème. Par contre, avec un facteur d'accélération de 6000, on tomberait à une différence de 10 millisecondes, donc pouvant potentiellement poser des problèmes.

De plus, comme indiqué précédemment, la norme Java n'impose pas que la classe `Instant` fournisse une précision inférieure à la seconde. Cela veut dire qu'une implantation de la JVM peut choisir de ne fournir des instants qui ne sont définis qu'à la seconde près, et tous les calculs de durée entre les instants n'être également précis qu'à la seconde près. Utilisant Mac Os X, j'ai pu observer que les machines virtuelles fournies par Oracle ont une classe `Instant` précise au centième de seconde. Mais selon les observations d'anciens étudiants travaillant sous Windows, les machines virtuelles disponibles sous ce système fournissent une classe `Instant` précise à la seconde près seulement. Pour la classe `AcceleratedClock`, cela pose un problème lorsqu'un temps Unix est converti en instant puis qu'une durée entre deux instants est calculée car les centièmes de secondes sont alors perdus lors de la conversion. Il faut donc créer des scénarios où les actions sont espacées d'au moins une seconde et ne pas utiliser des facteurs d'accélération qui produiront des délais sous la seconde.

Pour votre culture personnelle, sachez qu'il existe des horloges matérielles et des systèmes d'exploitation beaucoup plus précis dans l'ordonnancement des tâches, c'est à dire pouvant descendre au millionnième voire au milliardième de seconde. Ce sont des systèmes d'exploitation temps réel utilisés dans les applications temps réel.

creative commons ©J. Malenfant, 2026

B Scénarios de test

L'horloge accélérée donne les moyens de construire des scénarios de test temporisés, mais si on les utilise directement cela reste un exercice de programmation « manuelle » au sens où il faut récupérer des délais entre tâches et faire les appels explicites à des méthodes comme `scheduleTask` pour construire son scénario de test. Pour s'abstraire un peu plus de cette approche un peu fastidieuse, BCM4Java propose un outil complémentaire sous la forme de scénarios de test définis par la classe `TestScenario`.

B.1 `TestScenario` et étapes de test

Les scénarios de test temporisés proposés par BCM4Java sont pour l'instant assez simples dans leur conception. Chaque scénario est construit pour être exécuté relativement à une horloge accélérée partagée par tous les composants participant au test. Les scénarios de test supposent que les composants participants sont créés par des scripts de la machine virtuelle à composants, qu'ils gèrent leur cycle de vie comme dans une application standard, à ceci près qu'ils connaissent le scénario de test et vont en lancer l'exécution en parallèle avec leur fonctionnement « normal » pour que les actions qui leur incombe dans le scénario soient exécutées, comme nous le verrons plus loin. Par simplicité, tous les composants participant à un scénario de test vont connaître l'ensemble du scénario de test global mais devront se contenter de n'exécuter que leurs propres actions au sein de ce scénario.

L'exécution d'un scénario de test est fait pour produire des traces sur la sortie standard. Il n'y a pas d'outils d'exécution comme en JUnit, les traces produites doivent donc être programmées manuellement. La seule aide apportée par la classe `TestScenario` est la possibilité de fournir deux chaînes de caractères représentant un « message » à produire sur la sortie standard au début et à la fin de l'exécution du scénario de test.

La classe `TestScenario` définit un scénario de test comme une série d'actions à exécuter par un ensemble de composants participants. Chaque action est confiée à un et un seul participant et il est programmé à un instant i fixé à l'avance par rapport à un instant de départ s également fixé à l'avance tels que $i \geq s$. Pour créer un scénario de test, on instancie la classe `TestScenario` avec l'un de ses deux constructeurs en fournissant :

- une URI d'une horloge qui va servir à synchroniser toutes les actions de tous les composants participants sur la même référence de temps des instants [7] ;
- un instant de départ qui doit correspondre à l'instant de départ de l'horloge précédente ;
- un instant de fin qui doit également correspondre à l'instant de fin de l'horloge précédente ;
- un tableau d'étapes de test dont les instants d'exécution sont entre l'instant de départ et l'instant de fin ainsi qu'ordonnés de manière croissante.

Une étape de test est représentée par la classe `TestStep`, elle-même implantant l'interface `TestStepI`. Une étape de test est définie avec :

- l'URI du composant participant devant exécuter cette action [8] ;
- l'instant de l'exécution de cette étape dans le scénario de test ;
- une fonction de type `FComponentTask` qui doit être exécutée au sein du composant participant pour réaliser l'action attendue de cette étape de test.

La fonction à spécifier est de même nature que celle utilisée lorsqu'on soumet un tâche à exécuter à un composant par la méthode `AbstractComponent::runTask`. Optionnellement, on peut fournir les deux messages de début et de fin de scénario de test précédemment introduits.

B.2 Exemple : dépôt de données simple

Pour comprendre le fonctionnement des scénarios de test, un petit exemple complet d'un dépôt de données vous sera fourni. Dans cet exemple, le dépôt de données offre l'interface `SimpleDataStoreCI` (voir la figure 10) où apparaissent trois signatures :

7. Cette URI va être utilisée pour récupérer l'horloge sur le serveur d'horloge en appelant ce dernier sur son port entrant standard.

8. L'URI utilisée est celle de son port entrant offrant l'interface `ReflectionCI` et qui est retournée par la méthode `AbstractComponent::createComponent`.

creative commons ©J. Malenfant, 2026

```
public interface SimpleDataStoreCI extends OfferedCI, RequiredCI
{
    public Serializable put(String key, Serializable data) throws Exception;
    public Serializable get(String key) throws Exception;
    public Serializable remove(String key) throws Exception;
}
```

FIGURE 10 – Interface `SimpleDataStoreCI` de l'exemple du dépôt de données.

— `put` permettant de déposer une donnée `data` associée à la clé `key` ;
— `get` permettant de récupérer la donnée associée à la clé `key` ;
— `remove` permettant de retirer une donnée du dépôt associée à la clé `key`.

Lorsque plusieurs clients utilisent un tel dépôt pour échanger des données, le client qui veut récupérer une donnée doit attendre que le client qui dépose la donnée l'ait fait par un appel à `put` avant de faire son appel à `get`. En programmation séquentielle, un test fera d'abord le `put` puis le `get`. Mais lorsque les clients sont des entités parallèles (voire réparties) s'exécutant à leur propre rythme en utilisant leurs propres *threads*, il est plus compliqué de s'assurer que le `put` se fasse avant que le `get` puisse se faire sans erreur.

En supposant que cette application ait été entièrement programmée, lui faire exécuter un scénario de test va nécessiter d'abord de créer le scénario de test. Pour cela, il faut commencer par identifier les composants qui vont participer au scénario de test. Dans l'exemple du dépôt de données, les composants clients font les actions utiles, alors que le composant dépôt est passif. Ce dernier ne sera donc pas participant au scénario de test alors que les composants clients le seront. Pour qu'un composant puisse être participant à un scénario de test, il faut :

1. Qu'on puisse se référer à son identifiant dans le scénario de test.
2. Que le composant puisse avoir accès au scénario de test.
3. Que le composant de test lance l'exécution du scénario pour que s'exécutent en son sein les étapes de test dont il est responsable.

L'identifiant unique utilisé pour les composants est celui retourné par la méthode `AbstractComponent::createComponent`. Il est possible d'imposer cette URI lors de la création en lui passant cette URI en paramètre de son constructeur qui va appeler le constructeur de sa superclasse `AbstractComponent`. Cette méthode est la plus simple pour pouvoir se référer à un composant dans le scénario de test. On peut en profiter pour passer le scénario de test aussi via le constructeur de la classe du composant de la manière suivante :

```
protected DataStoreClient(String reflectionInboundPortURI, TestScenario testScenario)
{
    super(reflectionInboundPortURI, NB_THREADS, NB_SCHEDULABLE_THREADS);

    // Preconditions checking
    assert testScenario != null && testScenario.entityAppearsIn(
                                        getReflectionInboundPortURI()) :
           new PreconditionException(
               "testScenario != null && testScenario.entityAppearsIn("
               + "getReflectionInboundPortURI())");

    this.testScenario = testScenario;
    ...
}
```

On suppose ici que la classe `DataStoreClient` définissant les composants clients possède une variable d'instance `testScenario` dans laquelle le scénario de test sera rangé pour être exécuté le moment venu (voir le code fourni et sa documentation pour les détails de l'implantation). Le paramètre `reflectionInboundPortURI` est l'identifiant du composant et aussi l'URI de son port entrant offrant l'interface de composant `ReflectionCI`, comme nous le verrons en cours.

Dans le script de la machine virtuelle à composants, nous allons créer deux composant clients. Nous définissons deux URI pour cela :

```
/** component URI for the first simple data store client.  */
public static final String CLIENT1_URI = "simple-data-store-client-1";
/** component URI for the second simple data store client. */
public static final String CLIENT2_URI = "simple-data-store-client-2";
```

Pour le scénario de test, on va définir une méthode statique `testScenario` dans la classe `CVM` pour le créer et dans la méthode `deploy`, on crée les deux composants en leur passant leur URI et le scénario de test :

```
TestScenario testScenario = testScenario();
AbstractComponent.createComponent(DataStoreClient.class.getCanonicalName(),
                                  new Object[] {CLIENT1_URI, testScenario});
AbstractComponent.createComponent(DataStoreClient.class.getCanonicalName(),
                                  new Object[] {CLIENT2_URI, testScenario});
```

Avant de passer à l'exécution du scénario, développons le scénario lui-même. Pour cela, il faut définir une horloge, avec une URI qu'il sera facile de partager, un instant de départ, optionnellement un instant de fin et un facteur d'accélération. Dans la classe `CVM`, on ajoute pour cela les définitions de variables statiques suivantes :

```
/** for test scenarios a {@code Clock} is used to get a time-triggered
 *  synchronisation among the actions of components.                      */
public static String CLOCK_URI = "test-clock";
/** start virtual instant in the test scenario, as a string to be parsed.  */
public static String START_INSTANT = "2026-01-30T09:00:00.00Z";
/** end virtual instant in the test scenario, as a string to be parsed.    */
public static String END_INSTANT = "2026-01-30T09:02:00.00Z";
/** the acceleration factor applied to the {@code Instant} time
 *  reference to run the test faster or slower than real time.            */
public static double ACCELERATION_FACTOR = 60.0;
```

puis dans la méthode `deploy`, il faut créer les données utilisées pour créer l'horloge :

```
// the start time of the scenario is defined relative to the current time
long current = System.currentTimeMillis();
// start time of the components in Unix epoch time in nanoseconds
long unixEpochStartTimeInNanos = TimeUnit.MILLISECONDS.toNanos(current + DELAY_TO_START);
// start instant used for time-triggered synchronisation in the test scenario
Instant startInstant = Instant.parse(START_INSTANT);

// create the clock server and the clock used tp synchronise the
// components actions in the test scenario
AbstractComponent.createComponent(
    ClocksServer.class.getCanonicalName(),
    new Object[] {
        CLOCK_URI,       // must use the same in the test scenario
        unixEpochStartTimeInNanos,
        startInstant,    // idem
        ACCELERATION_FACTOR
        });
```

La constante `DELAY_TO_START` en millisecondes est fixée de telle manière à ce que tous les composants aient le temps d'être créés, démarrés puis prêts à faire ou recevoir des appels de service. Elle doit donc être ajustée selon la complexité du démarrage de l'application et la puissance de calcul de l'ordinateur qui influence la durée de cette phase initiale.

Un scénario de test très simple apparaît à la figure 11. Il s'agit de la méthode statique `testScenario` invoquée précédemment qui est dans la classe `CVM`. Cette méthode définit les instants de départ et de fin du scénario à partir des constantes statiques vues précédemment. Ensuite, elle déclare des valeurs qui vont servir dans les étapes de tests. Puis viennent la définition des instants où vont être exécutées les actions du scénario de test. Regrouper les définitions de ces instants permet de se rendre compte rapidement s'ils sont bien disposés dans le temps et bien entre l'instant de départ et l'instant de fin.

Ici, nous utilisons la méthode `Instant::plusSeconds` pour créer de nouveaux instants relativement à l'instant de départ. Cette approche fonctionne bien pour de petits scénarios. Pour des scénarios plus élaborés, il peut devenir plus lisible de définir les instants de manière absolue, à partir de chaînes de caractères comme nous l'avons fait pour les instants de départ et de fin. On peut aussi opter pour un mélange des deux, en définissant les instants majeurs du scénario de manière absolue puis les instants de chaque séquence majeure relativement à l'instant majeur de leur début. L'important est :

```
public static TestScenario    testScenario() throws VerboseException
{
    Instant startInstant = Instant.parse(START_INSTANT);
    Instant endInstant = Instant.parse(END_INSTANT);

    // test values
    String key1 = "x";
    Double value1 = 1000.0;

    // instant at which the first client component will make its put
    Instant putInstant = startInstant.plusSeconds(10);
    // instant at which the second client component will make its get
    Instant getInstant = startInstant.plusSeconds(20);
    // instant at which the first client component will make its remove
    Instant removeInstant = startInstant.plusSeconds(30);

    return new TestScenario(
        CLOCK_URI,         // URI of the clock used to synchronise
        startInstant,      // start instant on which actions times are based
        endInstant,        // end instant, the upper bound for actions times
        new TestStepI[] {
            new TestStep(
                CLOCK_URI,        // FIXME: should not have to be repeated
                CLIENT1_URI,      // URI of the component performing the action
                putInstant,       // instant at which the action will be executed
                owner -> {        // action, as a lambda expression
                    try {
                        ((DataStoreClient)owner).getOutPort().put(key1, value1);
                        ((DataStoreClient)owner).traceMessage(
                            " puts " + key1 + " with value " + value1 + "\n");
                    } catch (Exception e) {
                        System.err.println("put test failed with exception " + e);
                    }
                }),
            new TestStep(
                CLOCK_URI,        // FIXME: should not have to be repeated
                CLIENT2_URI,      // URI of the component performing the action
                getInstant,       // instant at which the action will be executed
                owner -> {        // action, as a lambda expression
                    try {
                        Serializable v = ((DataStoreClient)owner).getTestAction(key1);
                        assert value1.equals(v) : "get test failed!";
                    } catch (Exception e) {
                        System.err.println("gett test failed with exception " + e);
                    }
                }),
            new TestStep(
                CLOCK_URI,        // FIXME: should not have to be repeated
                CLIENT1_URI,      // URI of the component performing the action
                removeInstant,    // instant at which the action will be executed
                owner -> {        // action, as a lambda expression
                    try {
                        Serializable v = ((DataStoreClient)owner).getOutPort().remove(key1);
                        assert value1.equals(v) : "remove test failed!";
                        ((DataStoreClient)owner).traceMessage(
                            " removes " + key1 + " getting value " + value1 + "\n");
                    } catch (Exception e) {
                        System.err.println("remove test failed with exception " + e);
                    }
                })
            });
}
```

FIGURE 11 – Scénario de test simple pour l'exemple du dépôt de données.

— de s'assurer que les instants des actions s'enchaînement selon la logique du test (d'abord le `put`, puis le `get` et enfin le `remove`, si on définit un test qui doit fonctionner ; il est bien sûr à la fois possible et recommandé de construire des tests qui doivent échouer et lancer une exception) ;
— de s'assurer que les instants vont bien en ordre croissant ;
— que les délais entre les instants modulo le facteur d'accélération ne vont pas tomber sous la limite de précision de l'horloge du système d'exploitation (de l'ordre de 10 millisecondes

sous une JVM dont la classe `Instant` est précisé au centième de secondes, plutôt quelques secondes pour celles dont la classe est précise à la seconde près).

Ensuite, la méthode crée l'instance de `TestScenario` pour la retourner à l'appelant. Le constructeur de la classe `TestScenario` prend en paramètres :

— l'URI de l'horloge servant à synchroniser les actions du scenario,
— l'instant de départ,
— l'instant de fin,
— le tableau des actions à exécuter en ordre croissant des instants.

Chaque étape du scénario, correspondant à une action à exécuter par un participant au scénario, est définit comme une instance de la classe `TestStep`. Le constructeur de cette classe prend en paramètres :

— l'URI de l'horloge servant à synchroniser les actions du scenario,
— l'URI du composant devant exécuter l'action prévue à cette étape,
— l'instant auquel cette étape doit être exécutée,
— le code de l'action à exécuter sous la forme d'un lambda expression prenant en paramètre la référence à l'objet représentant le composant participant et dont le corps est le code à exécuter à cette étape.

Notre petit scénario de test pour l'exemple comporte trois étapes. À la première étape, le composant client 1 fait le `put` des données de test définies plus haut. À la seconde étape, le composant client 2 fait le `get` de la donnée avec la même clé. Enfin, à l'étape 3, le composant client 1 retire la donnée du dépôt. Notez les instructions `assert` qui servent d'oracle pour chacune des étapes. L'oracle est la condition à remplir pour considérer que le test passe ou non. L'utilisation d'oracles est la condition *sine qua non* pour avoir des tests automatisables, c'est à dire que l'on peut exécuter sans intervention humaine [9].

Revenons maintenant au composant client qui doit faire exécuter ses actions dans le scénario de test. Nous nous sommes arrêtés ci-haut au moment où le constructeur a défini son URI et récupéré le scénario de test dans sa variable locale `testScenario`. Le composant poursuit ensuite son exécution normale en faisant ses initialisations (termine l'exécution de son constructeur) puis son démarrage (exécution de sa méthode `start`). Pour lancer l'exécution de ses actions dans le scénario, il doit ajouter au code de sa méthode `execute` les éléments suivants :

```
// initialise the clock with the given URI, retrieving it from the clock server;
// this must be done by the component before running the test scenario
this.initialiseClock(ClocksServer.STANDARD_INBOUNDPORT_URI, this.testScenario.getClockURI());
// make the component execute its actions in the test scenario
this.executeTestScenario(this.testScenario);
```

Les méthodes appelées sont définies dans `AbstractComponent` comme support pour l'exécution des scénarios de test. La méthode `initialiseClock` se connecte au serveur d'horloge sur son port entrant dont l'URI est fournie (ici l'URI standard définie par la classe `ClocksServer`) et récupère l'horloge dont l'URI lui est fournie. La méthode `getClock` permet alors de récupérer la référence à l'horloge si besoin est. La méthode `executeTestScenario` prend en charge l'exécution progressive de toutes les étapes de test du scénario dont le composant lui-même est resposable de leur exécution. Cette méthode ordonnance sur le *pool* de *threads* ordonnançable standard du composant toutes les actions du scénario de test pour ce composant les unes après les autres en utilisant l'horloge pour savoir à quel moment en temps réel les tâches correspondantes doivent être déclenchées.

L'exécution du scénario de test (voir le code fourni pour comprendre les éléments complémentaires) apparaît à la figure 12.

9. Tous les projets de développement logiciel digne de ce nom aujourd'hui utilisent des tests automatisés, entre autres choses pour vérifier la non régression (pas de *bugs* apparaissant à l'issue du *commit*) lors des *commits*. Sur les dépôt de code sous git, un pipeline d'actions est généralement déclenché lors des *commits*, dont l'exécution de tests automatisés de non régression. En cas de test qui ne passe pas, le *commit* est rejeté. Il ne fait généralement pas bon effet pour un développeur d'avoir trop de ses *commits* rejetés, ce qui laisse généralement des traces visibles lors des négociations d'augmentation de salaire suivantes...

creative commons ©J. Malenfant, 2026

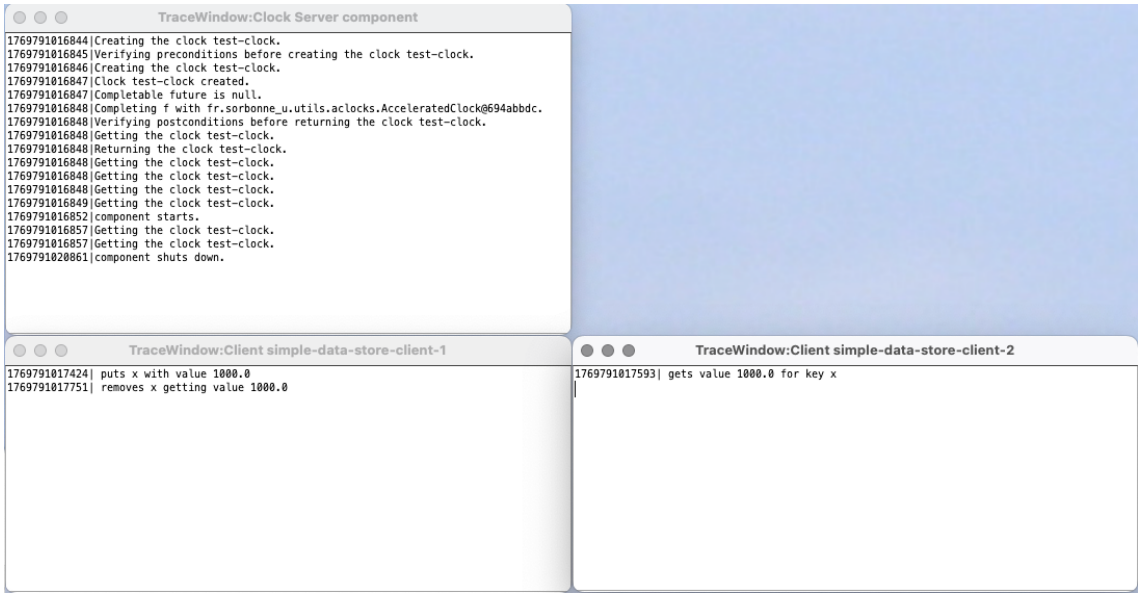


FIGURE 12 – Résultat de l'exécution du scénario de test tel qu'il apparaît dans les fenêtres de trace des composants.

B.3 Précautions à prendre

Outre les précautions sur lesquelles nous sommes revenus pour l'utilisation de l'horloge, l'exécution des scénarios de test ajoutent quelques difficultés potentielles supplémentaires. Comme on a pu le voir, l'exécution des actions de test se fait en parallèle avec l'exécution des autres tâches du composant. Il est donc possible de définir des scénarios de test où un composant exécute à la fois des actions de test prévues au scénario et d'autres tâches déclenchées par l'exécution du code de l'application. Cela devient toutefois plus délicat à bien organiser.

D'une part, le mélange temporel des deux types de tâches peut avoir un impact sur la temporisation des actions, en forçant le décalage de certaines actions faute de *threads* pour les exécuter au moment souhaité.

D'autre part, cela ouvre la possibilité d'exécution parallèle, requérant une gestion de concurrence entre les actions de test et le code du composant qui s'exécuterait en parallèle. Si le composant a été conçu pour s'exécuter avec du parallélisme interne, les problèmes d'exclusion mutuelle ont pu déjà être anticipés et résolus. Par contre, pour certains composants prévus pour s'exécuter sans parallélisme interne (totalement séquentialisés, par exemple), il faudra prendre des précautions supplémentaires :

— Soit le composant est modifié pour assurer l'exclusion mutuelle entre son code et les actions de test, ce qui n'est pas nécessairement souhaitable si dans son fonctionnement normal il n'est pas prévu qu'il y ait du parallélisme interne ou même cela peut modifier sa sémantique interne en brisant la séquentialité nécessaire.
— Soit on ne crée dans le composant qu'un seul *pool* de *threads* ordonnaçable avec un seul *thread* et on fait exécuter toutes les tâches, celles venant du code du composant et celles venant des actions de test sur ce même *pool* de *thread* qui assurera alors la séquentialisation complète de toutes les tâches, y compris les tâches correspondant aux actions du scénario de test,au prix de décalages possibles dans le temps des actions qui ne pourraient être exécutées en temps voulu si l'unique *thread* est occupé à cet instant là.

creative commons ©J. Malenfant, 2026